PLAN

Otonom Araç — full project plan

Dyl-Startech · MEB Uluslararası Robot Yarışması

Egemen Yusuf K. · Tuna B.

Danışman: Mehmet Emin U.

Snapshot built 02 August 2026

This PDF is a read-only snapshot. The living document is `PLAN_New.md` in the project repository — edit that, never this. If this file and the repository disagree, the repository is right.

New here? Read only section 0.0 (“Coming back after a break”). It is one page and it is the front door.

Tuna: read `Tuna.txt` instead — it is written for you.

Contents

PLAN — Dyl-Startech Otonom Araç

Single source of truth for the car. Written 1 August 2026. Revised 1 August 2026 against the official **Otonom Araç Kategori Kılavuzu 2026** (see section 2).

0. Read this first (for any agent or person picking this up)

0.0 Coming back after a break? Read ONLY this page

This document is a **reference, not a book.** Nobody reads it end to end — you read the section you need, when you need it. This page is the front door.

The five things to know before touching anything:

1. **The old code in `LEGACY/` is good.** PD steering with dynamic gain, adaptive lighting profiles, `gpiozero`, `picamera2`, event debouncing. Do **not** rebuild it blindly — several things this plan proposes "building" already exist there, better.
2. **May failed on unfinished calibration and unconnected features, not bad algorithms.** That distinction decides everything else.
3. **The perspective quad is stale** — 640×480 coordinates in an 800×680 frame, with a warning comment nobody actioned. It biases the car ~80 px sideways and hides the nearest road. Strongest single suspect. (§3.1)
4. **The motor trims were never measured**, and the tool that measures them prints variable names the config stopped reading. Fix the tool *before* trusting it. (§3.2, §20.3e)
5. **The old `.txt` documentation is fabricated** — invented constants, methods, filenames and metrics. Do not trust `BASLA_BURADAN.txt`, `DOSYALAR_GUNCELEME_DURUSU.txt`, `DEGISIKLIKLER_OZET.txt` or `IMPLEMENTATION_SUMMARY.txt`. (§21)

Do this first, from anywhere, no car needed — 15 minutes: commit `LEGACY/` unmodified and push to the VPS. Every finding above lives on one untracked disk until then. (§12, Phase 0)

First session with the car and a floor — four physical checks:

1. **Trace two motor wires.** Left/right or front/rear? Blocks everything; if it is front/rear the control law cannot work at all. (§15 q1)
2. Confirm the motors are on the L298N **OUT** pins, not IN.
3. Check what PWM is actually running — no frequency is set anywhere, so it defaults to 100 Hz. (§13)
4. Multimeter across the motor terminals at full duty. Suspected ~175% of rating. (§3.6)

Then, before committing to any rewrite: run the cheap experiment in §20.7 — fix the quad, measure the trims, drive `yol_takip.py`, read the stability report. One afternoon, and it tells you whether the old car was one measurement away from working.

Where to look when you need more:

If you are…	Read
Wondering why May failed	§3
About to write car code	§20 (what to keep), §4 (file list)
Wondering why the old docs lie	§21
Working on SUBIRU	§22
Checking rules, points, track layout	§2
Buying parts, competition day, practice track	§16, §17, §18 — not urgent in autumn

If you are Tuna: read `Tuna.txt`, not this file. It is written for you, in plain language, and it covers everything above.

This project has been worked on by several different AI assistants in separate sessions. Each one started blind, added its own files, and left. That is the main reason the codebase drifted. **This file exists to stop that.**

Rules for anyone continuing this work:

1. **Read this file before touching anything.** Also read `CLAUDE.md` (project rules) and `Tuna.txt` (plain-language change log).
2. **Do not create files that are not listed in section 4.** If you believe a new file is needed, add it to this plan first and say why. A new file is a plan change, not an implementation detail.
3. **No code for the *new* car has been written yet** — deliberately; Egemen wants the design settled first. But a complete previous implementation **does** exist in `LEGACY/`, and it is far more capable than this plan originally assumed. **Read section 20 before concluding that anything is missing.** Several things this plan proposes "building" already exist there in better form.
4. **When something here changes, update this file and append a line to `Tuna.txt`** explaining it in simple language.
5. Anything marked **[UNVERIFIED]** has not been physically checked. Do not build on it without confirming first.
6. **The category guide outranks this file.** Where they disagree, the guide wins and this file is wrong. Re-read the guide when the new edition is published (section 2.6).

1. The project

Two pieces in one repo:

- **The car** (`arac/`) — a Raspberry Pi 5, camera-only autonomous vehicle for the Uluslararası MEB Robot Yarışması, Otonom Araç kategorisi. The new build; not written yet.
- **`LEGACY/`** — the previous implementation, dated **4 May 2026**, two days before the competition. Complete and sophisticated. It is the reference implementation and the benchmark the new car must beat. See section 20.
- **SUBIRU** (`subiru/`) — a small Flask dashboard for tracking who owns what and whether it is progressing. Built, working, but currently has **zero tasks in it**. Name is short for "*Şununla bir uğraşsan*".

Team

Person	Current role (per technical document)
Egemen Yusuf K. (E.Y.K.)	Software development
Tuna B. (T.B.)	Hardware design
Mehmet Emin U.	Advisor teacher

School: Denizyıldızları Mesleki ve Teknik Anadolu Lisesi, Darıca / Kocaeli.

This division is being revised — see section 11. As written it gives Egemen 100% of the work, because a vision-only car is almost entirely software.

Team size: the guide allows **up to three students** plus a mandatory advisor teacher at high-school level. The team currently has two. A third student is permitted and would be genuinely useful — see section 11. At most **two students may be in the competition area at one time**, and the advisor may not take an active role there.

Working rhythm

No work happens over summer. Work happens during school term. This plan is therefore ordered by **sequence, not dates** — each phase must finish before the next starts. Per `CLAUDE.md`, do not assume deadlines exist. Section 12 sizes the phases against the term so the sequence stays honest about how much time there actually is.

2. The rules, in numbers

Everything in this section comes from the **Otonom Araç Kategori Kılavuzu 2026** and the 2026 Uygulama Kılavuzu. It is here because the previous plan was written from the team's own technical document rather than the guide, and several design decisions were being made without knowing what they were worth.

2.1 Scoring

Item	Points	Notes
Button start, no external computer	**50**	Awarded for capability, before the run
Correct start on green, 1st attempt	**50**	2nd attempt = 25. Nothing after that
Yaya geçidi (pedestrian crossing)	**50**	Stop within 30 cm, wait ≥ 5 s
Hemzemin geçit (level crossing)	**50**	Stop within 30 cm, wait ≥ 5 s
Hız tümseği (speed bump)	**50**	Cross without leaving track or lane
Araç sollama (overtaking)	**100**	Only inside the overtaking-permitted zone
Çıkmaz yol (dead end)	**100**	Detect, turn right, never enter
Park etme (parking)	**100**	**Red** bay only, fully inside
Bölge tamamlama (zone completion)	**50**	No track exit, no lane violation
Task total	**600**	
Finish-time coefficient	**240 − finish seconds**	Only if the course is completed

Three consequences the old plan did not account for:

Speed is worth roughly a task and a half. Finishing in 120 s adds 120 points — more than parking. Finishing in 200 s adds 40. The car cannot be tuned purely for safety; a slow, perfect run leaves a three-figure score on the table. The `base_speed` and the curve slowdown in section 6 are scoring parameters, not just comfort settings.

There is a hard 4-minute limit (240 s). Past that the referee ends the round. Points already earned stand, but the time coefficient is zero because the course was not completed. A car that dawdles at every task can run out of course.

Rounds are summed, not best-of. Total score is the sum across all rounds run. A car that scores 400 twice beats a car that scores 550 once and 150 once. This is an argument for *consistency over peak performance* — which pulls against the time bonus. The resolution: make the car reliable first, then raise `base_speed` in `ayarlar.json` only as far as the black-box log (section 10) shows it still completing cleanly.

2.2 Hard technical limits — these are disqualification conditions

- Must fit **easily inside a 20 × 30 cm box**. Maximum height **25 cm**.
- Wheel diameter **≤ 10 cm**. No weight limit.
- **Camera only.** Distance sensors, Lidar, IR, ultrasonic — any of them means elimination. **[ACTION]** Confirm the car carries no leftover HC-SR04 or IR module from an earlier build, including unconnected ones still bolted on.
- **No limit on the number of cameras.** See section 2.5 — this is an opportunity.
- No restriction on controller board, motor count or motor RPM.
- **Original software required.** Block-based or ready-made commercial control software (the guide names LEGO Spike's standard software) is banned. OpenCV as a library is fine; a downloaded self-driving project would not be. Whoever is at the table must be able to explain any part of the code on request.
- Bluetooth, Wi-Fi, IR and RF **off during the run**. Detection of an active module or remote control is immediate disqualification. This is the constraint that makes section 10 necessary.
- No liquid, flammable, explosive or pressurised energy sources. Battery packs are fine.

- A **QR code** is stuck to the car's fixed body at registration. Removing it, moving it or damaging it is disqualification. It must not be on a removable part. Plan a flat, permanent spot for it now, and do not run the mounting screws through it.
- Between rounds only **tyres and batteries** may be changed. Changing the body is disqualification. Electronic parts may be swapped only for the same type in the same position — and if the QR code is damaged doing so, disqualification.

[ACTION for Tuna, before anything else] Measure the car against the box. This is the cheapest possible failure to avoid, and it fails at technical inspection before the car ever reaches the track. Two Pi-mounted L298N boards front and rear plus a battery pack is not obviously under 30 cm long.

2.3 The track, precisely

Black surface. Solid and dashed road lines. Road edges bounded by **white lane stripes**. Cars must stay inside their own lane throughout.

The layout is: start area → task zones → overtaking zones → dead-end section → coloured parking areas → finish. **Blue roadside markers** define the zone-completion areas.

Objects on track:

- **Traffic light** at 1 m from the start, ±10% tolerance. It cycles red, yellow and green; the intervals are random. The car must move within 3 s of green.
- **Overtaking target: one orange vehicle, 20 × 30 × 25 cm.** Its position is reset by the referee for every competitor, and it is only ever placed in a zone where overtaking is permitted.
- **Decoys: yellow vehicles, 20 × 45 × 25 cm**, placed at random positions in **no-overtaking** zones, in the opposite lane, as visual obstacles.
- **Signs, all 13 × 20 cm**: yaya geçidi, hemzemin geçit, hız tümseği, sollama serbest, çıkmaz yol, park yeri. Each is aligned with the start of the feature it announces, at the track edge.
- **Parking**: three bays, red, blue and green. **Their positions change before every round.** Red is the target.

Orange versus yellow is the single most dangerous colour decision in the project. They are adjacent hues, they are both large blocks in the frame, and confusing them means either overtaking in a no-overtaking zone (failed task, likely lane violation) or ignoring the real target. This must be calibrated with both objects present under competition lighting, not with one of them in a classroom. It goes at the top of `kalibrasyon.json`.

2.4 The task order is not fixed

The positions **and the order** of the pedestrian crossing, level crossing and speed bump are set by the referees before each round, and **may be different for every competitor and every round**. Parking bay colours also move.

The old section 5 assumed a fixed sequence. It cannot. The state machine must handle these three tasks arriving in **any order**, which changes its design — see section 7.

2.5 Two cameras are legal

The guide places no limit on camera count. This is worth considering seriously once Phase 3 works, because the two jobs have opposite requirements:

- **Lane keeping** wants a downward, close, narrow view for a fast, stable error signal.
- **Signs and lights** want a forward, distant, wide view with enough resolution to classify a 13 × 20 cm sign in time to react.

One camera compromises both. Two cameras let each pipeline run at its own resolution and region of interest. `goz.py` is already specified as swappable-backend, so this is an extension rather than a redesign.

Do not do this in Phase 1 or 2. It doubles the calibration surface and the loop cost. It is a Phase 4+ option, listed here so `goz.py` is written to allow it rather than retrofitted later.

2.6 Dates and editions

The 18th competition ran **6–8 May 2026 in Antalya**, themed "Yeşil Vatan, Mavi Gelecek". The 2026 category guide was published around January 2026. So the working assumption for pacing is: **guide published ~January, competition ~May**, with applications opening before that.

This means the autumn term is spent building against the **2026** rules, and the new guide lands mid-year. Section 12 has a checkpoint for re-reading it. No category requires a robot design or production report for Otonom Araç — the guide states one is not requested — so document work is for the school's own purposes, not for judging. **[VERIFY]** Confirm this against the Uygulama Kılavuzu, which governs the application itself.

3. What went wrong at the May 2026 competition

Reported symptom: **"It failed to follow the road a lot."**

> **This section was rewritten on 1 August 2026, after the actual competition code was found in `LEGACY/`. The previous version was wrong in a dangerous way: it claimed the car had no proportional steering and called that "the main bug." That came from reading the team's *technical document*, which describes the car inaccurately. The real code has a well-developed PD controller. Anyone acting on the old section 3 would have spent months rebuilding a subsystem that already works.**
>
> **Lesson worth keeping: the technical document is not a reliable description of the car. Where they disagree, the code wins.**

3.0 What the May car actually had

Verified by reading `LEGACY/` (dated 4 May 2026, two days before the competition):

- **Proportional PD steering.** `KP = 0.30`, `KD = 0.45`, plus dynamic gain on large errors, derivative capping against oscillation, automatic slowdown into corners, dead-zone PWM compensation, and graceful decay when the lane is lost.
- **Adaptive lane detection.** Perspective warp to bird's-eye → CLAHE on the L channel → a white mask whose profile is selected by mean brightness (DARK / NORMAL / BRIGHT) → column histogram with continuity weighting → near/far weighted error.
- **`gpiozero`** (Pi 5 native) and **`picamera2`**, both import-guarded so the code falls back to webcam and no-op motors when run off the Pi.
- **Event debouncing** — a detection must persist for N frames before it fires.
- A per-frame error logger, and nine separate calibration and tuning tools.

The earlier theories — no proportional control, naive fixed thresholding, no simulation mode, no logging, RPi.GPIO on a Pi 5 — are **all false**. Strike them out.

Partly settled by the code — with a warning. `motor.py` drives exactly **two control channels**, named `LEFT` and `RIGHT`, and the car has **four motors, two paralleled per L298N channel**. What the code does *not* establish is which physical motors are on which channel — those names are labels chosen by a programmer, not a wiring diagram. The schematic suggests wires crossing between boards, i.e. front/rear. **Question 1 is reopened and blocking** (section 15). If the pairing really is front/rear, the control law in section 6 cannot work and that alone explains May.

Current-limit consequence of four motors: two motors share one L298N channel's ~2 A budget. Two 6 V gearmotors stalling together — which is exactly what happens on the speed bump — will exceed it, causing thermal shutdown or a brownout. And this is **worse than 2 A suggests**, because section 3.6's overvoltage raises stall current proportionally. Measure it before trusting it.

> **The pattern across 3.1–3.3: the code is good, the calibration was never finished, and three features were never connected.** That is a completely different failure mode from "the code is bad," and it changes what the rewrite is for. See section 20.

3.1 The perspective quad was never recalibrated — strongest candidate

`LEGACY/config.py` line 23 carries an unfixed warning:

```
# ⚠️  800×680 çözünürlük için yeniden kalibre edilmeli (calibrate.py çalıştırın).
PERSP_SRC = [[160, 300], [480, 300], [0, 480], [640, 480]]
```

`WIDTH = 800`, `HEIGHT = 680`. Those coordinates are from a **640×480** frame. Every other resolution-dependent constant *was* migrated — `ROAD_ROI_BOTTOM = 680` proves it. This one was flagged as needing recalibration and then left behind.

Two consequences:

- **An 80-pixel lateral bias.** The quad's horizontal centre is x=320; the frame's centre is x=400. The bird's-eye view is therefore not centred on the camera axis, but `lane.py` computes error against `bird_w // 2` as though it were. Working through the bottom edge of the warp, a perfectly centred car reads an error of roughly **−100 px**. With `KP = 0.30` that is a permanent correction of about 30 PWM applied to a car that is already where it should be. **The car veers, constantly, and no amount of PD tuning fixes it** — the controller is faithfully correcting an error that does not exist.
- **The nearest road is invisible.** The quad's bottom edge is at y=480 in a 680-row frame, so the closest 200 rows never enter the warp. The "near" histogram is actually mid-distance, which is made worse by `LANE_FAR_WEIGHT = 0.60` already weighting the far slice more heavily than the near one.

For scale: `ASSUMED_LANE_WIDTH = 300`, so the bias is about a third of a lane width.

[VERIFY — one afternoon] Run `calibrate.py`, fix `PERSP_SRC` for 800×680, and drive it. This is the cheapest high-value experiment available and it should be done **before** committing to the rewrite — see section 20.7.

3.2 The motor balance was never calibrated either

In `LEGACY/config.py`:

```
LEFT_TRIM_LOW   = 1.0
LEFT_TRIM_HIGH  = 1.0
RIGHT_TRIM_LOW  = 1.0
RIGHT_TRIM_HIGH = 1.0
```

All four at default. `LEGACY/motor_balance_test.py` exists for the sole purpose of measuring these, and `BASLA_BURADAN.txt` lists it as **step 1** of the tuning workflow. It was either never run, or its results were lost.

Cheap gearmotors are never matched. On a differential-drive car an uncorrected imbalance is a permanent pull to one side. The PD controller then spends the entire run fighting a systematic bias instead of following the road — and with `KI = 0.04` the integral term partly masks it on straights, then dumps the accumulated bias into the next corner.

[VERIFY — Egemen] Did the car pull consistently to one side? If yes, this is very likely the answer, and the fix is one afternoon with a tool that already exists.

3.3 Two latent trim bugs — masked now, will bite on first calibration

Both are invisible while the trims are `1.0`, because multiplying by one twice changes nothing. They activate the moment real values are measured.

Wrong wheel. `LEGACY/controller.py:_apply_speed_dependent_trim` selects the trim by the *sign* of the PWM value rather than by which wheel it belongs to:

```
trim = LEFT_TRIM_LOW if pwm >= 0 else RIGHT_TRIM_LOW
```

It is called once for the left wheel and once for the right. Whenever both drive forward, **both receive the LEFT trim.** A correction applied identically to both wheels cannot correct an imbalance *between* them — it only scales overall speed.

Applied twice. `controller.py` trims, then `motor.py:set_speed` trims the same value again, that time correctly and per-wheel. Net effect: `LEFT_TRIM²` on the left wheel, `LEFT_TRIM × RIGHT_TRIM` on the right.

`LEGACY/CLAUDE.md` records the double application as "*intentional in current code.*" That sentence is how the bug survived review, and it is the single clearest argument for the rewrite in section 20.

3.4 The physical start button was removed

`LEGACY/main.py`'s header states **"GPIO 16 buton kaldırıldı"**, and the documented start paths are typing `GG` or `EZ`, or pressing `SPACE`, read via `termios` / `tty` — which means a terminal, which means a computer attached to the car.

Two consequences:

- The guide awards **50 points** for starting from a physical button with no external computer (section 2.1). A keyboard start forfeits those points outright, and sits uncomfortably close to the no-external-computer rule.
- This is the same `GG` / `EZ` hotkey that `CLAUDE.md` names as its canonical example of a change to reject for having no good reason. It was not someone tampering with the code — it is in the competition build as the primary start path.

The button must come back. See section 5.

3.5 Dead-end detection never fired — 100 points of dead code

`LEGACY/main.py` line 356 reads the sign type:

```
sign = events.get('sign_type')
if sign == 'cikmazsokak':      →  ÇIKMAZSOKAK, brake
elif sign == 'sollamabam':     →  set the no-overtaking window
```

`events.py` never produces a `sign_type` key. Its `detect()` returns exactly `traffic_light`, `crosswalk`, `crosswalk_close`, `hemzemin`, `hemzemin_close`, `speed_bump`, `orange_car`, `yellow_car`, `parking_zone`, `sign_blue` — and nothing else. So `events.get('sign_type')` is **always `None`**, and both branches are unreachable.

Consequences:

- **Çıkmaz yol (dead end) — 100 points — could never be scored.** The handling state exists, is written correctly, and is never entered.
- **The no-overtaking zone never activates.** `_no_overtake_until` is only ever assigned inside the dead `sollamabam` branch, so overtaking is permitted everywhere on the track. The only remaining protection is the `not events['yellow_car']` check, which is real but is a fallback, not the rule the guide describes.

There is also a **trained sign classifier** in the repo — `sign_model.json` (241 KB) plus `train_sign.py` — and it is referenced *only* by `train_sign.py` and `sign_test.py`. It was built and never connected to the pipeline. That is almost certainly what was meant to supply `sign_type`.

So the model exists, the consumer exists, and the wire between them was never run. This is the single largest recoverable points loss found so far, and it is a plumbing job rather than an algorithm one.

This also resolves open question 14 in a useful direction: the ML-versus-CV argument is partly moot, because the ML side was never actually in the loop. Whatever May scored, it scored with classical CV only.

3.6 Overvoltage — compounds everything above

From the assembly photo: the L298N runs from **3× 18650**, so the 6 V motors see roughly 9–10.6 V, about **175% of rating**. `LEGACY/config.py` then sets `BASE_SPEED = 62` and `MAX_SPEED = 85`.

An overvolted motor is faster and more torquey than the PD gains were tuned for. Every correction lands harder than intended, which makes overshoot worse and tuning far more difficult. Combined with an uncalibrated imbalance (3.1), this is a car that is genuinely hard to keep on the road.

`max_pwm` should start near **57%**, not 85%. See section 6.

3.7 Still plausible, not yet evidence-backed

- **Dashed centre lines.** `lane.py` does estimate a lane centre rather than chase the brightest blob, so the original worry is partly answered — but whether dashed segments are handled cleanly has not been checked. **[VERIFY]** by replaying footage through it.

- **Venue lighting.** The adaptive DARK/NORMAL/BRIGHT profiles are a real defence, but they were tuned somewhere that was not Antalya. The guide explicitly refuses complaints about lit screens, scrolling text and camera flashes around the track.
- **The Antalya edits are lost.** Egemen modified the code at the venue and those changes were not saved. `LEGACY/` is the 4 May state, *not* what actually ran. Anything finer than the findings above cannot be recovered from this code.

3.8 There was still no way to find out

`logger.py` records only per-frame error, for a fixed duration. With radio off there is no SSH, no screen and no dashboard during a run, so the car came back with almost nothing to say. This is why `kayit.py` (section 10) is one of the few genuinely new things worth building rather than inheriting.

4. File layout — this is the complete list

```
arac/                  the car
  main.py              entry point, startup, the one main loop, button start
  durum.py             state machine (durum = state)
  goz.py               camera input, swappable backends (goz = eye)
  goruntu.py           vision: masks, lane finding, sign and light detection
  surucu.py            motor control, swappable backends (surucu = driver)
  ayar.py              loads and validates settings
  kayit.py             black-box run recorder (kayit = record)
  bildir.py            LED / buzzer status output (bildir = notify)      [NEW]
  ayarlar.json         tunable numbers (speeds, gains, ROI, timings)
  kalibrasyon.json     HSV colour ranges — owned by Tuna
arac.service           systemd unit, starts main.py on boot              [NEW]
requirements.txt       pinned dependency versions                        [NEW]
klipler/               recorded track footage for offline testing        [NEW]
subiru/                the monitoring dashboard (already exists)
LEGACY/                the 4 May 2026 implementation — read-only reference
PLAN.md                this file
CLAUDE.md              project rules
Tuna.txt               plain-language change log
```

`LEGACY/` is read-only — enforced by git, not by avoidance. It is the reference implementation and the benchmark (section 20), and the 4 May state must stay recoverable so that "is the new car better?" always has an honest answer.

That does **not** mean it can never be touched. Sections 20.7 and 21.7 deliberately require changing `PERSP_SRC`, fixing `motor_balance_test.py` and deleting the four fabricated documents. The rule is:

1. **Commit it unmodified first** (Phase 0, first bullet). Until that happens, nothing in `LEGACY/` may be edited at all, because there is no way back.
2. After that, changes are allowed, must be visible in a diff, and must be recorded in `Tuna.txt`.
3. Never run the **new** car from `LEGACY/`, and never fix a bug there instead of in the new code. Diagnostic changes only.

Naming follows the project convention of short Turkish names, matching SUBIRU.

Justification for the four additions (required by rule 2 in section 0):

- **`bildir.py`** — with radio off and no screen, the car currently has no way to say "config is bad" or "camera failed" before a run. See section 5. It is a genuinely separate concern from motor control and from recording, and it is about twenty lines.
- **`arac.service`** — section 5 explains why the car cannot be started by hand at the venue. A systemd unit is the mechanism.
- **`requirements.txt`** — the project already lost time to an RPi.GPIO/Pi 5 mismatch and an OpenCV version nobody can name. Pinning is what stops that recurring, and it is what makes a rebuilt SD card identical to the working one.
- **`klipler/`** — a directory, not a module. Phase 2 develops vision against recorded video on Windows; those clips are the regression test set and belong in the repo structure rather than on someone's desktop. Large files go to git-lfs or stay local with a manifest, decided in Phase 2.

5. Starting the car with no screen and no radio

This was missing entirely and it is worth 50 points plus the whole run.

Rule: the car must be brought to a ready state by a **button or similar trigger**, from software already loaded, **with no external computer connection**. That is the 50-point award. And under 2.3 there is no Wi-Fi, so there is no SSH to type `python main.py`.

So the boot path has to be:

1. Pi powers up. `arac.service` starts `main.py` automatically, with a restart policy.
2. `ayar.py` loads and validates `ayarlar.json` and `kalibrasyon.json`. **Motors stay off.**
3. `goz.py` opens the camera and confirms frames are actually arriving — not just that the device opened.
4. The car enters `BEKLE` and signals **ready**.
5. Button press → `ISIK_BEKLE`. Only now is the traffic light being watched.

Status output (`bildir.py`). Without a screen, a silent car is indistinguishable from a dead one. Minimum viable: one LED and one buzzer, or two LEDs.

Signal	Meaning
Slow blink	Booting, loading config
Solid	Ready, waiting for the button
Fast blink	Config invalid or camera not producing frames — do not press the button
Double beep on press	Button registered, now watching for green
Continuous	`HATA` — motors off, log flushed

Cheap, and it converts "why isn't it going" from a mystery at the table into a glance. An LED is not a sensor, so it is not affected by the sensor ban.

Battery. The guide states no extra time is given at the track for charging. Cells must arrive charged, with charged spares — battery swaps between rounds are explicitly allowed.

6. The control law — specification for the rewrite

Normal lane keeping uses **proportional differential steering**, not pivots.

> **Note:** this is a specification for the new code, **not** a list of missing features. `LEGACY/controller.py` already implements all of it and more (section 3.0). Read that file before writing this one — the intent here is to rebuild it understandably, not to invent it. The genuinely new requirements are the `max_pwm` ceiling in step 5 and the single-place trim rule.

Every frame:

1. From the region of interest, find the lane boundaries and estimate the **lane centre**. Where only one boundary is visible, infer the centre from it and the known lane width. Where the centre line is dashed and currently absent, this is normal — see 3.2.
2. Compute `error` = how far the estimated lane centre sits from the image centre. Negative means the lane is to the left, positive to the right.
3. Compute a steering correction from that error using a proportional term plus a damping term based on how fast the error is changing (a PD controller). The legacy values were `KP = 0.30`, `KD = 0.45` — start there rather than from zero.
4. Apply it as a *difference between the two sides*, with both sides still driving **forward**: - one side gets `base_speed - correction` - the other gets `base_speed + correction` - both clamped to a legal PWM range
5. Reduce `base_speed` when the error is large — slow down for sharp curves rather than trying to take them at full speed.

Pivot turns are reserved. Full counter-rotation is only used for the dead-end manoeuvre and, if needed, parking alignment. It is never used for lane correction.

Extras that matter:

- **Deadband** — ignore very small errors so the car doesn't twitch on a straight.
- **Slew limiting** — cap how fast PWM may change between frames, to protect the gearboxes and reduce current spikes.
- **Line-lost behaviour** — if both boundaries disappear, do *not* immediately stop or spin. Hold the last correction briefly (the lane usually left the frame in a known direction), then slow, then search. Record the event via `kayit.py`.
- **Asymmetry trim** — a constant offset in `ayarlar.json` correcting the two sides not being equally fast at equal PWM. They never are. Without it, `Kp` is being tuned to compensate for a mechanical bias, which is why gains stop working after a repair.

The maximum PWM clamp is set by voltage, not by 255. [UNVERIFIED — measure it]

From the assembly photo (August 2026), the L298N is fed by **3× 18650**. That is 11.1 V nominal and 12.6 V fully charged. The L298N drops roughly 1.5–2 V across its bridge, so the motors see approximately **9–10.6 V**. The motors are 6 V.

That is roughly **175% of their rated voltage at full duty.** Consequences:

- 100% PWM is not "fast", it is abusive. It shortens gearbox life and raises current draw.
- It made every tank pivot more violent than intended, which would have amplified the oscillation in section 3.1. This is a contributing cause of the May failure, not just a hardware detail.
- `max_pwm` **in** `ayarlar.json` **should start around 57%** (6 ÷ 10.6), not 100%. Treat the range above it as headroom that exists but is not used.

Verify before relying on this. Put a multimeter across the motor terminals at full duty, with charged cells. If it reads near 10 V the arithmetic holds. If the motors turn out not to be 6 V, recompute. This is a five-minute measurement that sets a number every other tuning decision depends on — do it in the first Phase 1 session.

All gains (`Kp` , `Kd`), base speed, clamps, deadband, slew limits and trim live in `ayarlar.json` . None of them are hardcoded.

7. State machine (`durum.py`)

> **Every state name in this section is PROPOSED for the new car. None of them exist in any code today.** `LEGACY/main.py` uses eleven different names — `BEKLIYOR`, `SURUYOR`, `YAYA_YAKLAS`, `YAYA_GECİDİ`, `HEMZEMIN_YAKLAS`, `HEMZEMIN`, `TUMSEK`, `SOLLAMA`, `PARK`, `CIKMAZSOKAK`, `PARK_TAMAM` (section 21.3). Do not grep for the names below and conclude the code is broken; do not cite them as though they were built. This warning exists because a proposal being mistaken for a description is precisely what section 21 documents.

Section 4.2 of the technical document checks all seven tasks against every frame in priority order. This breaks on a real track: once the run has started, any red object in the crowd re-triggers "traffic light," and any white-striped object re-triggers "pedestrian crossing."

Use an explicit state machine instead. In each state the car watches for a small number of specific transitions and ignores everything else.

But not a fixed sequence. Per section 2.4 the referees reorder the crossings and the bump for every competitor. So the design is:

- `SERIT_TAKIP` is the hub. Everything returns to it.
- While in `SERIT_TAKIP` the car watches for **the signs of all not-yet-completed tasks simultaneously** — this is the one place multi-detector work is unavoidable.
- Each task state is entered from the hub, does one thing, and returns.
- Each completed one-shot task is marked done so it cannot re-trigger.

State	What it does	Leaves when
`BEKLE`	Motors off, waiting for the physical start button	Button pressed
`ISIK_BEKLE`	Stationary, watching only for a green light	Green seen → move within 3 s
`SERIT_TAKIP`	Lane keeping; watches for remaining task signs	A task trigger fires
`GECIT_DUR`	Stopped at pedestrian / level crossing	≥ 5 s elapsed, then resume
`TUMSEK`	Reduced speed over the speed bump	Bump cleared
`SOLLAMA`	Overtake: change lane, pass, return	Manoeuvre complete
`CIKMAZ`	Dead-end detected, turn right before entering	Turn complete
`PARK`	Find the **red** bay among three, align, stop inside	Stopped
`BITTI`	Motors off, recorder flushed	Terminal
`HATA`	**Motors off.** Any uncaught error lands here	Terminal

Notes tied to the rules:

- `GECIT_DUR` **must stop within 30 cm of the crossing and wait at least 5 s.** With no distance sensor, "30 cm" has to be inferred from where the crossing sits in the frame. That mapping is a calibration value, measured once with a tape measure and stored in `ayarlar.json`. Wait 6 s, not 5 — the margin is free and a short stop scores nothing.
- `SOLLAMA` **must confirm it is inside a "sollama serbest" zone before moving out.** The trigger is the *sign*, not the orange vehicle. Seeing an orange block is not permission. A yellow vehicle is never a reason to change lane.
- `CIKMAZ` **must turn before entering.** Entering the dead end fails the task even if the car recovers.
- `PARK` **chooses by colour, not position.** The bays move every round.

8. Vision pipeline (`goruntu.py`)

Shared per-frame pipeline, in this order:

1. Grab frame from `goz.py`.
2. **Downscale** to a small working resolution. This is the single biggest speed win.
3. **Crop to a region of interest.** For lane keeping, the lower portion of the frame (near the car). Use a second, higher band for lookahead so curves and signs are seen early. ROI bounds live in `ayarlar.json`.
4. Convert to HSV.
5. Apply the mask needed by the **current state only** — plus, in `SERIT_TAKIP`, the sign detectors for tasks not yet completed. Never all detectors unconditionally.
6. Clean the mask (morphological open/close) before contour analysis.
7. Return a small, plain result describing what was seen — not raw images.

Per-task detection approaches carry over from the technical document (sections 4.3 and 6): green/red segmentation for the light, contour analysis for crossing stripes, yellow-black striping for the bump, orange blob plus sign for overtaking, sign shape for the dead end, red/blue/green rectangles for parking.

Two additions from the guide:

- **Signs are all the same size (13 × 20 cm) and at the track edge.** Their apparent size is therefore a usable distance estimate, and their position is a usable filter — a candidate in the middle of the road is not a sign.
- **Blue markers denote zone-completion areas.** Blue is also a parking bay colour. If both use the same HSV entry they will confuse each other; give them separate entries in `kalibrasyon.json` from the start even if the values begin identical.

All HSV bounds come from `kalibrasyon.json`. **No colour value is ever written in a `.py` file.**

9. Configuration and calibration

Two separate JSON files, deliberately split by owner:

- **`ayarlar.json`** — behaviour: speeds, PID gains, ROI, loop rate, timings, pin numbers, camera backend choice, log verbosity, trim, frame-to-distance mapping. Owned by Egemen.
- **`kalibrasyon.json`** — HSV upper/lower bounds per colour and object. Owned by Tuna.

`ayar.py` loads both, validates them, and fails loudly with a readable message if a value is missing or out of range — and signals it via `bildir.py`, since there is no screen. A bad config must never reach the motors.

A small Flask calibration screen (same toolkit as SUBIRU) shows the live camera feed with sliders for each HSV bound and writes `kalibrasyon.json`. This satisfies the `CLAUDE.md` requirement that every option be configurable through a GUI, and it is what makes calibration a job Tuna can own without opening Python.

Calibration at the venue has a time limit. The guide gives **deneme turları** — practice rounds with equal time per team. That is the only window to see the real track under the real lights. It is short. So:

- The calibration GUI must be usable in **minutes, not an hour**. Sliders per colour, one save button, no restart required to apply.
- Wi-Fi is only forbidden **during a run**. During practice and in the pit a laptop is presumably fine — **[VERIFY] with the referees on the day**, and have a wired fallback (HDMI screen plus keyboard, or a phone via USB tethering) in case it is not.
- Bring `kalibrasyon.json` under version control so a bad on-site tune can be reverted in one command.
- Save a **timestamped copy** of `kalibrasyon.json` into every run folder (section 10), so each run's log states which calibration produced it.

10. Black-box recorder (`kayit.py`)

Because radio must be off during a run (rule 2.3), the car has to record its own story to the SD card.

Per frame, append one line: timestamp, current state, estimated lane centre offset, computed correction, both PWM outputs, and any event fired. Additionally save a snapshot frame periodically and **always** on a state change or a lane-lost event.

Written to a run-stamped folder, together with a copy of both JSON config files. Buffered, flushed on state change, and flushed in the `HATA` state so a crash still leaves evidence.

This is what makes the next bad run diagnosable instead of mysterious, and it is the only honest way to confirm a fix actually worked.

Cost control. Writing every frame at full rate to an SD card can itself slow the loop. Log lines are cheap; images are not. Cap snapshot frequency in `ayarlar.json`, and measure loop time with recording on and off during Phase 3 so the recorder is never the reason the car is slow.

11. Division of labour — the revision

The document's split ("Tuna: hardware, Egemen: software") is why Egemen ended up with everything. The revision below gives Tuna a real, attributable subsystem.

Tuna B.	Egemen Yusuf K.
Chassis, wiring, power, motor mounting	Architecture and the main loop
20 × 30 × 25 cm compliance and QR mounting spot	`goruntu.py`, `durum.py`, `surucu.py`
`kalibrasyon.json` and on-site colour tuning	Control law and tuning of gains
Track footage recording for replay testing	`kayit.py` and diagnosis
Physical start button, status LED and buzzer	`bildir.py`, `arac.service`, boot path
Charged batteries and spares on the day	The build
Running the car during tests, trying to break it	

Deliberate design point: Tuna's work makes Egemen's work faster and better, but never blocks it. Footage improves the vision pipeline, but Egemen can bootstrap with a phone video. Calibration values matter enormously on site, but defaults exist. If Tuna stalls, the project slows — it does not stop. Egemen's critical path must never depend on someone else's task being finished.

A third student is allowed and there is a clean subsystem for them. The obvious carve-out is **track and test**: build a practice track from the guide's dimensions, run the regression clips, keep the run log spreadsheet, own the pre-run checklist on the day. It is real, it is separable, and it is the work that otherwise silently lands on Egemen at 11 pm the night before. Second choice: own the parking task end to end (100 points, well isolated, only runs at the end of a lap).

Seed these into SUBIRU. `subiru/data/tasks.json` is currently `[]`. The task list in section 12 should be entered into the dashboard, assigned by owner, so the tool that exists for exactly this purpose finally has something in it. Also update `subiru/owners.py` — it still says `["T.B.", "E.Y.K."]` and should carry real names.

12. Build order

Sequence, not dates. Each phase finishes before the next begins. Each phase now has an **exit test** — a specific thing that either passes or does not. "It seems better" is not an exit test.

Phase 0 — Make the project survivable (first week back)

The cheapest phase and the one that protects every other one. *Updated 1 August 2026 after the `LEGACY/` audit.*

- **Commit `LEGACY/` first, before anything else.** All 26 files, unmodified. Every finding in sections 3, 20 and 21 rests on them, and they currently exist on **one disk, untracked, with no remote**. This is now the highest-priority item in the plan: the evidence is worth more than the plan that describes it.
- **Push to a private git remote — the team has a VPS, so use it.** Flagged as the most valuable missing protection since before summer. It is a task now, not a wish.
- On the VPS: `git init --bare ~/ototot.git` — a bare repo is just storage, nothing to host, nothing to secure beyond SSH.
- Locally: `git remote add origin <user>@<vps>:ototot.git`, then push `master`.
- Add Tuna's SSH public key so he can push calibration values from **his own machine**. This matters for section 11 — his contributions become attributable without needing Egemen's laptop, which is the difference between a real division of labour and a bottleneck.
- No GitHub account required, no free-tier limits, no privacy question. `LEGACY/CLAUDE.md` refers to a GitHub remote (`egdmte/ototot`) that this repo does not have; the VPS sidesteps whether that account still exists.
- **This is what finally makes `CLAUDE.md`'s "delete the folder and clone it again" STOP banner actionable.** Until now there has been nowhere to clone from.
- **Nightly backup of what git should not hold.** The VPS can also take the large binaries a repo should not carry: the SD card image, track footage for `klipler/`, the `.docx`. A scheduled `rsync` costs nothing and covers the corrupt-card-before-competition scenario this plan already calls fatal.
- Commit the rest of what is uncommitted: `CLAUDE.md`, `Tuna.txt`, `PLAN_New.md`, `run.bat`, the `subiru/app.py` port change. (`AGENTS.md` was deleted on 1 Aug — see section 19.)
- **Fix `motor_balance_test.py`'s output** so it prints the four names `config.py` actually reads (`LEFT_TRIM_LOW/HIGH`, `RIGHT_TRIM_LOW/HIGH`) instead of the two dead ones. Do this **before** Phase 1 runs it, or Phase 1's first measurement will silently do nothing again — see section 20.3e.
- **Lay out the Stage 1 tape track** (section 18.2). A roll of tape, and it unblocks Phases 2 and 3.
- `requirements.txt` with pinned versions — `opencv-python`, `numpy`, `picamera2`, `gpiozero` — plus the Python version and OS image recorded.
- **Image the working SD card** once the Pi boots correctly, and keep the image. A corrupt card two days before the competition is otherwise fatal.
- Answer the open questions in section 15. Most take minutes. **q1 is reopened and blocks Phase 1** — tracing two motor wires is the single highest-priority physical check in the project, because if the pairing is front/rear the control law cannot work at all.
- **Fix `LOG_DURATION_SEC`** — it is `120` against a 240-second race (section 21.6). One number, and it is the difference between having evidence from the back half of a run and not.
- Seed SUBIRU with these tasks.

Exit test: the repo can be cloned onto a second machine and the environment rebuilt from `requirements.txt` without anyone remembering anything.

Why `LEGACY/` must be committed before it is touched. Section 4 calls `LEGACY/` read-only, but sections 20.7 and 21.7 require changing `PERSP_SRC`, fixing `motor_balance_test.py`, and eventually deleting four documents. Those instructions contradict each other only while the folder is untracked. **Once it is in git, "read-only" is enforced by history rather than by not touching it** — any change is visible in a diff and revertible, and the 4 May state stays recoverable forever. That is the real reason this bullet comes first.

Phase 1 — Prove the car can steer

Nothing else matters until this works.

- Run `LEGACY/motor_balance_test.py` and **record the trim values** — this was never done, and section 3.1 names it as the most likely cause of the May failure
- Confirm the motor count physically **[UNVERIFIED — section 15, q6]**
- Measure the voltage at the motor terminals at full duty, and set `max_pwm` from it (section 6)
- Confirm the car fits 20 × 30 × 25 cm and wheels are ≤ 10 cm **[section 2.2]**
- `surucu.py` with mock and real backends, motors-off default, built on `gpiozero` (already proven on this car — see section 20; do **not** spend time migrating off RPi.GPIO, the real code never used it)
- Trim applied in **exactly one place**, per wheel — the bug in section 3.2 exists because it was applied in two
- `ayar.py` and a first `ayarlar.json`
- Drive the car by hand at various left/right PWM splits, **wheels off the ground**
- Then on the floor: confirm it drives a smooth arc, not a pivot

Exit test: commanded (60, 80) makes the car trace a repeatable smooth arc on the floor, in both directions, three times running, with the asymmetry trim recorded.

Phase 2 — See the lane

- `goz.py` with USB, Pi Camera and video-file backends
- Tuna records track footage — straights, curves, **dashed sections**, both crossings, the bump, and both the orange and yellow vehicles, under at least two lighting setups
- `goruntu.py` lane detection, developed on Windows against recorded video
- Calibration GUI and `kalibrasyon.json`

Exit test: on a held-out clip the pipeline reports a sane lane-centre offset in **at least 95% of frames**, including through dashed-line gaps, without the operator adjusting anything mid-clip. Fixed clip set, measured number, written down.

Phase 3 — Close the loop

- The PD control law from section 6
- `bildir.py`, `arac.service`, and the full no-screen boot path from section 5
- `kayit.py` recording every run
- Tune gains on the real track until the car follows a lane smoothly at speed

Exit test: the car completes three consecutive laps of a plain lane loop with **zero lane violations** and no human intervention, powered on by button alone with no laptop attached. Log the lap times — this becomes the baseline the time bonus is measured against.

Phase 4 — Tasks, one at a time

Each one fully working and recorded before starting the next. Order by points per unit of difficulty, using section 2.1:

1. **Traffic light start** (50 + 50 = 100) — highest value, lowest difficulty, and it is a precondition for every timed run anyway.
2. **Pedestrian and level crossing** (50 + 50) — the same detector and the same behaviour twice. One implementation, two scores.
3. **Speed bump** (50) — mostly a `base_speed` reduction; the risk is getting stuck.
4. **Parking** (100) — well isolated, runs only at lap end, easy to rehearse.
5. **Dead end** (100) — sign detection plus a reserved pivot.
6. **Overtaking** (100) — last. It is the only task requiring a deliberate lane violation, the only one with a decoy object, and the only one where a mistake makes things worse than skipping it.

Exit test per task: ten attempts, at least eight scoring, black-box log reviewed for all ten.

Phase 5 — Full runs

End-to-end laps under varied lighting, **with the crossings and bump reordered between runs** to prove section 7 does not secretly depend on a sequence. Review the black-box log after every single run.

Exit test: five consecutive full runs, each under 240 s, each scoring above a target the team sets from Phase 4 results.

Deferred and optional — a C# WinForms run analyser

Recorded so it is not forgotten, and fenced so it cannot quietly become September's project.

What. A Windows desktop tool for post-run forensics: scrub a `kayit.py` run on a timeline, plot lane error and both PWM outputs, and show the snapshot frame the car was looking at **synchronised to the point on the curve**. That is the question every track session ends with — *what was it seeing when it went wrong* — and May could not answer it.

Why it qualifies when nothing else does. It runs on Windows where the work happens; it never touches the car or the competition code, so it carries no originality or explainability burden; it does not duplicate SUBIRU (forensics, not task tracking); and it attacks the project's single biggest gap.

The honest note. The technical case is thin — Flask plus matplotlib would do this in a language already in the project. If this gets built in C#, the real reason is wanting to build a WinForms project, which is a perfectly good reason and must be *written down as that one*. A reason stated up front is fine; a reason invented afterwards is how section 21 happened.

Conditions, all required:

1. **Not before Phase 3.** `kayit.py` must exist and have produced real logs first, or the data format is guesswork and gets rewritten.
2. **Never on the critical path.** No car task may depend on it.
3. Read-only over the logs. It never writes to `config.py` or `tasks.json`.
4. If the four Phase 1 hardware checks are still open, this does not start. The car steering outranks the tool for looking at why it does not.

The risk being fenced against: this project's signature failure is starting something competently and stopping one step short. A new language is the most enjoyable possible way to not fix `PERSP_SRC`.

Term shape

The phases are sequenced, not dated, but they are not weightless. A realistic reading:

- **Autumn term** — Phases 0, 1 and 2. Phase 2 is the long one; it is also the one that can be done at home on a laptop against recorded clips, so it survives weeks where the car is not accessible.
- **New guide published (~January)** — stop and re-read it. Diff it against section 2 of this file. Rules change; the committee explicitly reserves the right.
- **Spring term** — Phases 3, 4 and 5. Phase 4 is naturally interruptible, one task per working session, which suits short school-day sessions.

If time runs short, this is the order to sacrifice: overtaking first, then dead end, then the speed bump. Never sacrifice Phase 3. A car that keeps its lane, starts on green and stops at crossings scores 250 plus a time bonus and finishes the course. A car with a half-finished overtake manoeuvre and no reliable lane keeping scores close to nothing.

Every session, first ten minutes: `git pull`, read the last five lines of `Tuna.txt`, check SUBIRU for what is assigned.
Every session, last ten minutes: commit, push, append a line to `Tuna.txt`. This is what makes a one-hour school session productive instead of a re-orientation exercise.

13. Testing and safety

Non-negotiable, from `CLAUDE.md`:

- **Motors default to off** at startup, and on **any** uncaught error. Never assume a previous safe state. The `HATA` state exists for this.
- **The first test of any new motor-control code happens with the wheels off the ground or blocked** — never on the floor. Direction and speed logic is exactly the kind of thing that is wrong the first time.
- Flag this explicitly, out loud, before anything spins for the first time.
- Test under more than one lighting condition, always. A threshold that works in one room is not calibrated, it is lucky.
- **Keep the regression clips.** Any change to `goruntu.py` is re-run against the full `klipler/` set before it goes on the car. This is what stops a fix for one lighting condition quietly breaking another.

Stopping a car that is already moving

The plan had no answer to this, which is a gap — motors-off defaults protect against a car that hasn't started, not one that is already driving away from you.

With radios off there is no remote kill. So the procedure is physical and everyone should know it before the first floor test:

Pick the car up and switch the battery holders off. Lift first — a car in the air can do no damage regardless of what the software thinks. Then cut power.

Stop immediately if the car is smoking, has hit something and is still driving, is heading somewhere it shouldn't at speed, or — the one people hesitate over — **if you simply don't understand what it is doing.** Not understanding is sufficient reason. Hesitation is the actual danger, so the rule is deliberately generous.

Never run a driving car on a table. Floor only, with space around it. A fall breaks the chassis, the camera mount and the calibration in one go.

Batteries are measured under load, not at rest. A tired 18650 reads a healthy 4.1 V on the bench and collapses under motor current. This is a classic cause of a car that works in the workshop and fails halfway through a run — and it presents exactly like a software bug, which is how a week disappears. Measure with the motors running.

PWM note — a real problem, but verify the fix before applying it.

`LEGACY/motor.py` constructs `PWMOutputDevice(LEFT_PWM_PIN)` with **no frequency argument**, and `LEGACY/config.py` has no `PWM_FREQ` at all. gpiozero defaults to **100 Hz**. The older CascadeProjects build set `PWM_FREQ = 1000`; that setting was lost in the rewrite.

100 Hz is coarse for motor control, and it may explain `DEAD_ZONE_MIN_PWM = 30` — a dead-zone floor is exactly the workaround you reach for when low duty cycles produce no usable torque. Raise the frequency and the dead zone may shrink on its own. **Test that before inheriting the dead-zone hack into the new code.**

Two cautions on the suggested `dtoverlay=pwm-2chan` fix, both to be checked on the Pi rather than taken on trust — **[VERIFY]**:

1. **Bare `dtoverlay=pwm-2chan` maps to GPIO18/19, not GPIO12/13.** This car uses 12 and 13 (physical 32/33). Getting hardware PWM there needs the pin parameters, roughly `dtoverlay=pwm-2chan,pin=12,func=4,pin2=13,func2=4`. Adding the bare line would enable hardware PWM on pins nobody uses, leave the car on software PWM, and **feel like it had been fixed** — the precise failure the warning was meant to prevent.
2. **The overlay alone may not change what gpiozero does.** `PWMOutputDevice` drives software PWM through the pin factory; the overlay exposes hardware PWM via sysfs (`/sys/class/pwm/`), which gpiozero does not use. Enabling the overlay and *actually getting* hardware PWM are two different claims, and the second does not follow from the first. Driving the hardware channels may require a different library.

The underlying concern is sound — software PWM jitters under load and you can lose a week blaming the gains. But confirm what is actually running before changing gains, e.g. by checking whether `/sys/class/pwm/pwmchip0` exists and whether anything is using it.

Library note — already solved, do not redo. `RPi.GPIO` does not work on the Raspberry Pi 5, which routes its pins through a new chip (RP1) the classic library cannot reach. **The legacy code already uses `gpiozero`**, so this problem was solved before May and the new build should simply inherit that choice. The only thing still outstanding is that the *technical document* wrongly lists RPi.GPIO — see section 14.

Camera note: `cv2.VideoCapture` will not open the Pi Camera Module V2 ribbon camera on current Pi OS — that needs `picamera2`, which the legacy code already uses. `VideoCapture` is for USB cameras. The team plans to switch to USB as primary with the Pi Camera as a secondary backend, but a CSI ribbon is currently connected — see section 15, q7.

Risk register

Robotics projects fail on the boring things. In rough order of how much damage each does:

Risk	Cost	Mitigation
SD card corrupts	Total loss	Image the card after Phase 0. Keep a written copy.
Repo only exists on one laptop	Total loss	Private remote, Phase 0.
Car fails the 20 × 30 × 25 box	Never competes	Measure in Phase 1, not in Antalya.
A leftover ultrasonic/IR module is spotted	Elimination	Strip and check in Phase 1.
Motors brown out the Pi mid-run	Random unexplained failures	Separate supply rails, section 15 q5.
QR code on a removable part or damaged	Disqualification	Pick a flat fixed spot in Phase 1.
Batteries not charged on the day	No run at all	Charged spares; no charging time is given.
L298N or motor fails in the week before	Weeks lost	Buy a spare L298N and spare motors early.
Orange/yellow confusion	Failed overtake, lane violation	Calibrate with both objects present.
New guide changes the rules	Rework	January checkpoint in section 12.
Only one person can run the car	Single point of failure	Pre-run checklist, section 12, third student.

Spares to have before the competition, not after: one L298N, one set of motors, one SD card imaged and tested, one spare USB camera, charged battery cells, and the cables. A spare that has never been tested in the car is not a spare.

14. Corrections needed in the technical document

- **"Python 3.19"** — does not exist. Versions run 3.12 → 3.13 → 3.14. The dev venv in this repo runs **3.13**.
- **"16850 Şarj Edilebilir Li-Po"** — should be **18650**, and 18650 cells are Li-ion, not Li-Po.
- **Section 5.1 pin table** — pin 11 appears twice, once as "Sol motor ileri" and once as "Ön tarafın +". The Ön/Arka labelling contradicts the left/right control implied by section 5.2. Egemen confirms the real car is **left/right paired**, with the two L298N boards physically mounted front and rear of the chassis. The table should describe left/right control and stop mixing in board position.
- **RPi.GPIO** listed for a Pi 5 (section 3.1) — see section 13.
- **OpenCV 4.2** is from 2019/2020. State whatever version is actually installed, and pin it in `requirements.txt`.
- **Section 4.2's "check all seven tasks every frame"** — replaced by the state machine in section 7. The document should describe the machine, not the priority list.
- **Section 5.2's five movements** — the document should describe proportional differential steering, with pivots named as a reserved special case. This is the change that fixes the actual bug and it should be visible in the write-up.
- **Task count** — the guide defines **eight** tasks. Bölge tamamlama is missing from the team's document entirely.

15. Open questions

Each now has an owner and a phase, because unowned questions do not get answered.

#	Question	Owner	Needed by
1	**REOPENED 1 Aug 2026 — blocking.** Was closed on the grounds that `LEGACY/motor.py` names its two channels `LEFT` and `RIGHT`. **That reasoning was wrong:** identifiers are labels, not wiring. The code proves only that there are *two* channels, not which motors hang off them. The schematic appears to show motor wires **crossing between boards**, which would mean front/rear pairing — and if so the control law in section 6 does not work as written (a front/rear car cannot steer differentially). Physical evidence outranks a variable name. **Trace two wires.**	Tuna	**Blocks Phase 1**
2	Does the car fit 20 × 30 cm, under 25 cm tall, wheels ≤ 10 cm?	Tuna	Phase 1
3	Are there any non-camera sensors still physically on the car, connected or not?	Tuna	Phase 1
4	**Largely answered — and badly.** `LEGACY/main.py` says "*GPIO 16 buton kaldırıldı*" and starts on `GG` / `EZ` / `SPACE` via a keyboard. So the 50-point button start was probably **not** being earned in May. Remaining question: is the button still physically on the car, or was it removed too? See 3.3.	Tuna	**Phase 1**
5	**Battery configuration** — *partly answered by the August 2026 photo.* The Pi runs from **2× 18650 through a DC-DC stepdown regulator**, and the L298N from a separate **3× 18650** pack. Separate rails, so the brownout risk is largely already handled — good design that had not been written down. **Still open:** measure the actual voltage at the motor terminals at full duty. See section 6, motors appear to be driven at ~175% of rating.	Tuna	**Phase 1, first session**
6	**How many motors does the car have?** The photo shows two yellow gearmotors clearly, but there are two L298N boards, which implies four. The software drives two channels either way (q1), so this no longer blocks — but it decides whether motors are paralleled per side, which affects stall current and the L298N's 2 A per-channel limit.	Tuna	Phase 1
7	**Camera: CSI now, USB intended — this is a DECISION, not an error to be corrected.** The hardware currently has a CSI ribbon and the legacy code uses `picamera2`; that is the *present state*. Egemen stated the *intent* to move to USB with CSI kept as a secondary backend. Those do not conflict, and an assistant "correcting" the plan to match the hardware would be overwriting a human decision with an observation. **Only Egemen closes this.** Either way `goz.py` carries three backends (USB, picamera2, video file), so the decision changes which is the default, not the architecture.	Egemen	Phase 2
8	**Partly answered.** `LEGACY/lane.py` estimates a lane centre via column histogram with continuity weighting — it does *not* chase the brightest blob. Still open: how cleanly it handles **dashed** segments. Answerable by replaying footage through it. See 3.5.	Egemen	Phase 2
14	**Classical CV or ML for signs?** `CLAUDE.md` answers **CV**, but `LEGACY/` contains a trained `sign_model.json` and `train_sign.py`. The plan and the code disagree. Decide before Phase 4. See section 20.3.	Egemen	Phase 4
9	**Is bölge tamamlama 50 points total, or 50 per zone?** The guide says "birden fazla bölge" but awards "50 bölge tamamlama ödül puanı". If it is per zone it may outweigh every other task and reorders all of Phase 4. Ask the coordinators.	Advisor	Phase 4
10	May a laptop be connected to the car in the pit and during deneme turları? Section 9 assumes yes. Confirm on the day and carry a wired fallback.	Egemen	Competition
11	Is a third student joining, and which subsystem do they own? See section 11.	All	Autumn term
12	**2027 competition date, guide publication and application deadline**, once announced. The 2026 cycle was: guide ~January, applications closed **20 March**, competition **6–8 May**. See section 17.1.	Advisor + Egemen	**1 January 2027 reminder**

#	Question	Owner	Needed by
13	What documents does the application require, and what is the **kura kaydı** step? Find out in January, not March.	Advisor	January

16. Parts, budget and purchasing

Money is not the constraint here. Lead time is. Everything on this list is cheap relative to the project. The failure mode is not "we couldn't afford it", it's "it arrived in April and we'd already lost three weeks."

The rule that sets every date below: a spare that has never been fitted and tested in the car is not a spare, it is a souvenir. Parts must arrive early enough to be *installed once, tested, and removed again*. That is what makes them useful at 9 am in Antalya.

16.1 Already on the car — verify, don't assume

Confirm each still works in September before buying anything. Some of these have sat in a cupboard all summer.

Raspberry Pi 5 · USB camera · Pi Camera Module V2 on a CSI ribbon (see q7) · 2× L298N · motors · chassis · **DC-DC stepdown regulator** · 2× 18650 holder (Pi) · 3× 18650 holder (L298N) · SD card

16.2 Needed for the plan as written

Not spares — the plan does not work without these.

Item	For	Order by
Status LED, buzzer, resistors	`bildir.py`, section 5	September
Momentary push button	Start button, if q4 says it doesn't exist	September
Second SD card	The imaged backup, Phase 0	September
Tape measure or calipers	The 20 × 30 × 25 check	September
Multimeter, if the school hasn't one	Measuring motor voltage, section 6	September

A buck converter was on this list and has been removed. The August 2026 photo shows a DC-DC stepdown regulator already fitted, with the Pi on its own 2× 18650 pack separate from the motors' 3-cell pack. The brownout problem was already solved; it just wasn't written down anywhere.

16.3 Spares — order by February, not April

Item	Why	Priority
1× L298N	Most likely component to die, and it kills the car	High
1 set of motors	Gearboxes strip. Slew limiting helps, doesn't prevent	High
1× SD card, imaged and **booted once**	Card corruption is the classic total loss	High
1× USB camera, same model	A different model means recalibrating everything	High
Charged battery cells	Guide gives no charging time at the venue	High
Jumper wires, connectors, heat-shrink	Always the thing that fails	Medium

Same model matters for the camera. A different sensor has a different colour response, and `kalibrasyon.json` would need redoing — at the venue, in your practice slot, under pressure. Buy the twin.

16.4 Optional, Phase 4 and later

- **Second camera** (section 2.5). Legal, potentially a real gain, but doubles the calibration surface. Only if Phase 3 is comfortably done before the January tripwire.
- Better wheels or tyres. Allowed to be changed between rounds; grip affects how well the control law's assumptions hold.

16.5 Getting the money

Price this list on Robotistan, Direnç or Robotzade and fill in the column yourself — component prices move too fast for anything written in August to still be true.

Give the priced list to the advisor by October. Not because the parts are needed in October, but because school procurement is slow and a request submitted in February arrives in April. If the school won't fund it, October also leaves time to find another route. February does not.

17. Competition day and the application

17.1 The application — the deadline that ends everything

From the 2026 cycle:

Event	2026 date
Category guide published	~January
Application deadline	**20 March 2026, 18:00** (an extension was announced 14 March)
Competition	6–8 May 2026, Antalya

So the working assumption for 2027 is **guide in January, applications close in March, competition in May**. Confirm against `robot.meb.gov.tr` — do not rely on this table.

Three things about that deadline:

It does not depend on the car working. Register as soon as applications open. There is no advantage to waiting to see whether Phase 4 is going well, and every week of delay is a week closer to forgetting.

It lands in the worst possible place. March is the middle of Phase 4 and near exam season. It is exactly when a small team has the least attention spare. Do not rely on noticing it.

All required documents must be uploaded by the deadline — submitting the form alone is not enough. There is also a separate **kura kaydı** step on the site. Find out what both require in January, not March.

Owner: the advisor teacher, with Egemen as backup. Both, deliberately. A single owner for the one irreversible deadline in the project is a bad design.

Do this now, in August: set two reminders. **1 January 2027** — check `robot.meb.gov.tr` for the new guide and application dates. **1 March 2027** — hard check that the application is submitted and documents uploaded. Put them somewhere that will still exist in six months, not in your head.

17.2 Roles at the table

Only two students may be in the competition area at once. The advisor may not take an active role there. Decide the split in advance, because deciding it at the table costs the time you need for something else.

- **Driver** — places the car in its lane, presses the start button, performs technical interventions when the referee allows one. Hands on the car, nobody else's.
- **Spotter** — watches the run, notes what failed and where, tracks the 4-minute clock, talks to the referee. Never touches the car.

If a third student joins, they are in the stands with the charged spares and the laptop, and they are the one who copies the black-box log off between rounds.

17.3 The practice round is your only calibration window

The guide gives **deneme turları** with equal time per team. That is the one chance to see the real track under the real lights, and it is short. Go in with a written order, because you will not have time to explore:

1. **Orange versus yellow** first. It is the most dangerous confusion in the project (section 2.3) and the most likely to differ from your test room.
2. **White lane lines** — the venue's glare on black flooring is the thing that broke the threshold in May.
3. **Red parking bay**, and check it isn't confusable with the traffic light's red.
4. **Green light**, then **blue zone markers**.

Commit `kalibrasyon.json` to git before you touch it and again after, so a bad on-site tune is one command to undo. Copy it into the run folder so every log says which calibration produced it.

Also: **look at the track before your run and note the task order.** The referees reorder the crossings and the bump for every competitor (section 2.4). Knowing the order won't change the code, but it tells the spotter what should happen next, which is the difference between "it failed" and "it failed at the second crossing."

17.4 What to pack

Car · charged batteries **and** charged spares · spare L298N · spare motors · spare imaged SD card · spare USB camera · laptop with the repo cloned · **HDMI cable, small screen, keyboard** (in case a laptop connection isn't allowed — section 15 q10) · screwdrivers and hex keys · multimeter · jumper wires · heat-shrink and tape · the printed checklist below · a printed copy of the category guide · tape measure

The screen and keyboard are the item people leave behind and then need. Without Wi-Fi and without a permitted laptop link, they are the only way to see anything the car is doing.

17.5 Pre-run checklist — print this

Run through it before **every** round, including practice. It is short on purpose.

```
□  1. Wi-Fi, Bluetooth and RF OFF — verified on screen, not assumed
□  2. No sensors on the car except cameras — including disconnected ones
□  3. QR code present, undamaged, on the fixed body
□  4. Car fits 20 × 30 cm, under 25 cm, wheels ≤ 10 cm
□  5. Batteries charged; charged spares in the bag
□  6. Latest kalibrasyon.json loaded and committed
□  7. Previous run's log copied off the SD card
□  8. SD card has free space for this run's log
□  9. Camera lens clean; nothing blocking the view
□ 10. Wheels spin freely, no cable fouling
□ 11. Power on — LED goes SOLID (ready). Fast blink means STOP, config is bad
□ 12. Press start button — double beep heard
□ 13. Task order noted from looking at the track
□ 14. Driver and spotter agreed, both know their job
```

Items 1 to 4 are disqualification conditions. Item 11 is the one that saves a wasted run.

18. The practice track

Phase 3 is "tune the car until it follows a lane smoothly." Phase 5 is "full runs under varied lighting." Neither is possible without something to drive on, so **the track is the hidden dependency for the whole second half of the project** — and the most likely reason Phase 3 slips past the January tripwire.

18.1 The mistake to avoid

Do not try to build the competition parkur. It is large, it needs a room nobody will give you, and waiting for it will cost you the autumn.

Each phase needs a different amount of track, and the amounts are wildly different:

Phase	What it actually needs
2 — vision	**Footage.** Not a track. Video of a lane, filmed once.
3 — closed loop	**A lane.** Straight, a curve each way, a dashed section. No task objects.
4 — tasks	A short approach lane plus **the one object** for the task being built.
5 — full runs	The closest thing to a real parkur you can manage. Only now does scale matter.

Phases 2 and 3 are where the risk lives, and they need the least. Build for them first.

18.2 Stage 1 — tape on the floor, September, nearly free

White electrical or masking tape on the darkest floor available. Wrong dimensions, ugly, temporary. It does not matter.

Lay out: **a straight, a left curve, a right curve, and one stretch with a dashed centre line.** A closed loop if the room allows, because then the car runs continuously and nobody has to keep picking it up. A rounded rectangle with different-radius curves is better than a perfect oval — it exercises more of the control law.

This unblocks Phase 2's footage and all of Phase 3. **Do it in the first week back.** The good track can arrive in November; the risk cannot wait that long.

Film the footage with the car's own camera, at the car's own height. Phone video from standing height has a completely different perspective, and the region of interest tuned against it will not transfer. `PLAN.md` previously said Egemen could bootstrap with a phone video — true for getting started, but every ROI number must come from camera-height footage. Mount the camera on the chassis and push the car by hand.

18.3 Stage 2 — printed segments, November

For Phase 4 and 5, dimensions start to matter — the 30 cm stopping distance, the lane width, the sign sizes.

Recommended: matte vinyl banner, printed to the guide's dimensions, in segments. Turkish print shops do *branda baskı* cheaply by the square metre, and a printed track is more accurate than anything you can tape by hand.

Two specifications that are not optional:

- **Matte, never glossy.** Glare on a dark surface is literally failure cause 3.3. A glossy banner would actively make your threshold problem worse than the real competition.
- **Segments, not one sheet.** Separate straights, curves, and task mats. They roll up for storage, they fit through doors, and — most importantly — **they can be rearranged.**

That last point is worth more than it sounds. The referees reorder the crossings and the bump for every competitor (section 2.4). Rearrangeable segments are the only way to actually test that your state machine doesn't secretly depend on a sequence, which is a Phase 5 exit condition.

Get the exact dimensions from Şekil 2–10 of the category guide. They are figures in the PDF, not text — someone has to open it and read them off. Lane width, stripe spacing, sign sizes, parking bay sizes. Do this before ordering anything printed.

18.4 Task objects — cardboard, mostly

None of this needs to be well made. It needs to be the right size and the right colour.

Object	Make it from
Orange vehicle (20 × 30 × 25 cm)	A cardboard box, painted or papered orange
Yellow decoy (20 × 45 × 25 cm)	Same, yellow. **Build both, they must be distinguishable**
Signs (13 × 20 cm)	Printed on paper, glued to card, on a stick
Traffic light	**A phone held up, showing full-screen red / yellow / green.** Free, and you can randomise the intervals like the real one does
Pedestrian / level crossing	White tape stripes on the black surface
Speed bump	Foam pipe insulation or a wooden batten, yellow-and-black taped
Parking bays	Coloured card rectangles, red / blue / green
Blue zone markers	Blue card at the roadside

Build the orange and the yellow vehicle at the same time, from the same materials, and photograph them together under both lighting conditions. They are the most dangerous confusion in the project (section 2.3) and the pair is what you calibrate against. One without the other is useless for that.

18.5 Space, storage and who owns it

- **A room you can leave it in, or storage you can roll it into.** This is an advisor task — it needs someone who can ask the school for space. Raise it in September, not November.
- **Lighting you can change.** Section 13 requires testing under more than one lighting condition. A room with both windows and overhead lights, tested by day and after dark, satisfies this for free.
- **Owner:** the track belongs to Tuna's column in section 11 — it sits naturally with footage recording and physical testing. If a third student joins, this is their subsystem (section 11).

18.6 Cost and timing

Price the vinyl printing locally; it is sold by the square metre and moves too fast to write down here. Everything else is tape, cardboard and paint.

- **September** — Stage 1 tape layout. Secure the room. Read the dimensions off the guide.
- **October** — price the printing, put it on the advisor's list with the parts (section 16.5).
- **November** — printed segments arrive, before Phase 3 tuning gets serious.
- **February** — build the remaining task objects as Phase 4 works through them.

19. Where the conversation left off

Decisions already made and applied:

- Identity and access checks were **removed** from `CLAUDE.md` (the Android Studio fingerprint and the swimming question). Reason: the team tests on many machines, the checks cost time on every one, and there is no attacker — the historical incident was a legitimate teammate making a bad change, which is a code-review problem that git already solves, not an access problem.
- `Tuna.txt` was created; it did not previously exist.
- The GUI toolkit question is settled: **Flask**, matching SUBIRU.
- The repo has git but **no remote**. Pushing to a private remote is the single most valuable protection still missing — it is now Phase 0, not a wish.

Added in this revision (1 August 2026), after reading the official category guide:

- Section 2, the rules in numbers: point values, hard limits, track objects, and the fact that **task order is randomised per competitor** (2.4).
- Section 3.2, a second likely cause of the May failure: the car was probably following a **dashed** line and treating every gap as a lost line.
- Section 5, the no-screen boot path — `arac.service`, `bildir.py`, and how the car is started at all when there is no Wi-Fi and no monitor.
- Exit tests for every phase, a Phase 0, and a term shape with a January rule-change checkpoint (section 12).
- A risk register and a spares list (section 13).
- Owners and deadlines on every open question (section 15).
- Section 16, parts and purchasing: what to buy, and the point that **lead time, not money, is the constraint** — spares must arrive early enough to be tested, so February not April.
- Section 17, competition day: the **20 March application deadline** from the 2026 cycle and the two reminders that protect it, roles at the table under the two-student limit, a calibration order for the practice round, and a printable pre-run checklist.
- Section 18, the practice track: the point that **each phase needs a different amount of track**, a nearly-free tape layout for September that unblocks Phases 2 and 3, and rearrangeable printed segments later — matte, never glossy.

Read from the assembly photo (August 2026), all marked `[UNVERIFIED]` until measured:

- The L298N runs from **3× 18650**, so the 6 V motors are seeing roughly 9–10.6 V — about **175% of rating**. `max_pwm` should start near 57%, not 100% (section 6). This is also a likely contributing cause of the May failure, since it made every pivot more violent than intended.
- The Pi has its **own 2× 18650 pack and a DC-DC stepdown regulator**, separate from the motors. The brownout risk was already solved; nobody had written it down. The buck converter has come off the shopping list.
- A **CSI ribbon** is connected, which contradicts the "USB as primary" decision in section 13. New question 7.
- Only two gearmotors are clearly visible against two L298N boards. New question 6 — if it is a two-motor car, the blocking question 1 dissolves.

Added after the legacy code was found (1 August 2026):

- **Section 3 was rewritten entirely.** The old diagnosis was wrong; see the notice at the top of that section. `AGENTS.md` has since been deleted, so `CLAUDE.md` is the only rules file.
- **Section 20, Inheritance** — the decision to rewrite, and what carries over.
- Old blocking question 1 (motor pairing) is **closed**: `LEGACY/motor.py` drives two channels, left and right.

Immediate next step: **Phase 0.** Push to a remote, then mine `LEGACY/` per section 20.

20. Inheritance — what carries over from `LEGACY/`

Decision (Egemen, 1 August 2026): the car is being rewritten, not repaired.

20.1 Why rewrite

Not because the legacy code is bad — sections 3.0 shows it is genuinely sophisticated. The reasons are:

1. **The guide requires that any part of the code can be explained on request.** Code assembled by several LLMs across sessions, which nobody can now defend line by line, is a rule exposure at the judging table, not merely a maintenance problem.
2. `LEGACY/CLAUDE.md` **documents the double-applied trim as "intentional."** That is the project recording that it no longer knows why its own code does what it does.
3. **Egemen wants control of the code.** That is a legitimate goal in itself, and it is the thing that makes every future debugging session cheaper.

20.2 The rule: rewrite the structure, inherit the knowledge

The expensive thing in `LEGACY/` is not the files — it is the **discovered facts**. Someone learned, painfully, that a single white threshold does not survive changing light; that CLAHE is needed before masking; that events must be debounced or they false-trigger; that these motors do not turn at all below about 30% PWM. Each of those cost a failed test. **Throw the code away. Do not throw those away.**

20.3 Triage

Legacy	Verdict	Reason
`lane.py` pipeline	**Inherit the approach**	Warp → CLAHE → adaptive profile → histogram → near/far error is genuinely good. Port it stage by stage, understanding each. This is the file you will be asked to explain
`controller.py` algorithm	**Inherit, minus trim**	PD + dynamic gain + derivative cap + curve slowdown is sound. Trim is a motor-hardware concern and must live *only* in the motor layer — that missing split is what created the bug in 3.2
`motor.py` structure	**Inherit, fix two things**	Clean `gpiozero` wrapper. Fix the wrong-wheel trim selection; convert the hardcoded direction inversion into a config flag that names the wiring fact it compensates for
`events.py` debouncing	**Inherit the concept**	Requiring a detection to persist N frames is the main defence against false triggers. Review the individual detectors separately
`sign_model.json`	**Keep the file, decide the approach**	241 KB of trained model, expensive to reproduce. But `CLAUDE.md` answers "classical CV vs ML" with **CV**, and this is ML. Contradiction needs an explicit decision — new question 8
`config.py` + `config_to_be_migrated.py`	**Drop both**	Two config files, one named "to be migrated," and `tune.py` rewrites Python source via regex to save values. Replaced by `ayarlar.json` + `kalibrasyon.json` (section 9)
The nine tuning scripts	**Consolidate into one**	`tune.py`, `hsv_tune.py`, `pd_tune.py`, `calibrate.py`, `kalibrasyon.py`, `camera.py`, `camtester.py`, `motor_balance_test.py`, `sign_test.py`. This is the sprawl in its purest form. Section 9 already specifies one calibration GUI, owned by Tuna
`main.py`	**Rewrite**	The most tangled file, and where the missing button and the `GG` start live
`logger.py`	**Rewrite as `kayit.py`**	Records only per-frame error for a fixed duration. Needs to become the full black box (section 10)
`yol_takip.py`	**Drop**	Documented in `LEGACY/CLAUDE.md` as an abandoned experiment

20.3b Full-file audit — corrections to the triage above

Reading every remaining file changed several verdicts.

`yol_takip.py` **— DROP was wrong, keep it.** `LEGACY/CLAUDE.md` calls it an abandoned experiment. It is actually a 372-line **lane-following-only** build of `main.py` with event detection stripped out. That makes it the ideal harness for testing the `PERSP_SRC` fix in isolation, with no task logic to confuse the result. Use it in 20.7 step 3.

`camtester.py` **is misnamed.** Despite the name it needs no camera — it is an interactive **motor** test (`w` / `s` / `a` / `d` /space). Useful, and the name should change in the rewrite.

`kalibrasyon.py` **is already a consolidation attempt** — 599 lines of "run every calibration tool from one menu". The new single calibration GUI should start from its menu structure rather than from nothing.

`config_to_be_migrated.py` **— drop confirmed, with a reason.** Same stale `PERSP_SRC`, same `1.0` trims, older gain multipliers (`1.0` where `config.py` has `1.3` / `1.2`), and its Turkish comments are **mojibake** (`TÃ¼m` — UTF-8 read as Latin-1). It is a corrupted older copy containing nothing unique.

`guncelle.sh` **confirms a remote existed** (`origin/main`). The current repo has none.

20.3c The `sign_type` fix is ~20 lines and it is the best-value work in the project

`sign_test.py` demonstrates the complete working path already: find a blue blob → crop → classify. The classifier is **HOG features (128-dim) plus nearest-neighbour** — classical computer vision with a lookup table, not deep learning, so it raises no originality concern and question 14 largely dissolves.

And the labels line up exactly:

```
sign_model.json classes:
  ['cikmazsokak', 'hemzemin', 'kasis', 'park', 'sollamabam', 'yayagecidi']

main.py expects:
  'cikmazsokak'  and  'sollamabam'
```

The model was trained *for* that consumer, with matching spelling. `events.py` already has `_detect_sign_blue()` locating blue signs — it just returns a bool instead of the crop.

So the work is: return the bounding rect, crop it, classify, put the label in the events dict as `sign_type`. That unlocks **çıkmaz yol (100 points)** and the no-overtaking-zone logic, and the model covers all six sign types, not only the two `main.py` currently reads.

Caveat: 192 vectors from roughly 5–7 source images per class, ×6 augmentations. It will work but it is thin — more training photos are a good, bounded, well-defined job for Tuna.

20.3d The documentation is fabricated — this is the real disease

`BASLA_BURADAN.txt`, `DOSYALAR_GUNCELEME_DURUSU.txt` and `DEGISIKLIKLER_OZET.txt` (all dated 2026-04-25) are LLM-generated optimisation reports. **Essentially every checkable claim in them is false:**

Claimed	Actual
`KP: 0.40 → 0.60`	`KP = 0.30`
`KD_LARGE_ERROR_MULT ×1.5`	`1.2`
Derivative cap `±600`	`DERIV_CAP = 150`
config / lane / controller = 252 / 205 / 153 lines	293 / 253 / 184
`events.py` 600+, `main.py` 900+ lines	406 and 627
`config.py` has `ADAPTIVE_HSV_ENABLED`, `HSV_BRIGHT_THRESHOLD`, `HSV_DARK_THRESHOLD`	None exist — thresholds are hardcoded `100` / `200` inline
`lane.py` has `_get_adaptive_hsv()`	No such method

Claimed	Actual
Read `FINAL_DELIVERY.md`, `OPTIMIZASYONLAR_DETAYLI.md`, `YARISMA_GUNU_REHBERI.md`, `OZET_VE_DEGISIKLIKLER.md`, `README_OKUMA_SIRASI.md`	**None of the five exist**
`start.sh`	Does not exist
"±40px → ±15px deviation, +62.5%" · "24-26 → 28-30 FPS" · "+85-95 puan"	Never measured. Adding per-frame CLAHE **cannot** raise FPS

`BASLA_BURADAN.txt` even signs off with `📍 DOSYA KONUMU: /mnt/user-data/outputs/` — an AI sandbox path, copied verbatim into the team's repo.

This is the mechanism that sank the project, and it is not the code. A confident document asserted the car deviated ±15 px at 28–30 FPS and was "✅ TÜM OPTİMİZASYONLAR UYGULANMIŞTIR". Nobody measured any of it. Its own pre-race checklist — step 1, motor balance — was never completed, which we know because the trims are still `1.0`.

Two consequences for the rewrite:

1. **Delete all three files.** They are worse than useless; they are confidently wrong. Nothing in them survives into this plan except this warning.
2. **Rule for the new project:** a performance number may only be written down if it came out of `kayit.py` or `logger.py` on a real run, with the date of that run beside it. Predicted improvements are not results, and an LLM's estimate of a competition score is not a measurement.

20.3e Why the trims stayed at 1.0 — probably not laziness

`motor_balance_test.py` ends by printing:

```
📋 config.py'ye şunu yapıştır:
   LEFT_TRIM  = 0.95
   RIGHT_TRIM = 1.0
```

But `config.py` reads **four** variables: `LEFT_TRIM_LOW`, `LEFT_TRIM_HIGH`, `RIGHT_TRIM_LOW`, `RIGHT_TRIM_HIGH`. The tool emits two names that nothing reads.

`DOSYALAR_GUNCELEME_DURUSU.txt` confirms how: `config.py` was given speed-dependent trim profiles while `motor_balance_test.py` was explicitly left `ORIJINAL`. The tool and the config it writes to were changed independently and never reconciled.

So the test may well have been run, its output pasted in, and nothing happened — a dead variable sitting in `config.py` while the four live ones stayed at `1.0`. **Fix the tool before trusting it** (20.7 step 2).

20.4 Mine these before deleting anything

`DEGISIKLIKLER_OZET.txt` is 22 KB of change history — precisely the "clear history" the legacy is said to lack. It may already record what was tried and what failed. `BASLA_BURADAN.txt` holds the tuning workflow in the correct order. Both should be read and their **facts** extracted into this plan before they are discarded.

Specific values worth rescuing:

- The perspective-warp corners and the working HSV profiles.
- `DEAD_ZONE_MIN_PWM = 30` — below this the motors stall.
- The event debounce frame count.
- `CAMERA_BGR_OUTPUT` and `CAMERA_ROTATE_180` — these exist because some Pi 5 and libcamera combinations return BGR despite being asked for RGB, and because the camera is mounted upside down. Rediscovering that costs an afternoon of confusion.

20.5 The convention that stops the rot returning

`LEGACY/` decayed for one specific reason: **nobody could say why a line was there.** The new code gets one rule, which is `CLAUDE.md`'s "every action needs a reason" applied to constants:

> Every non-obvious number traces to a measurement, and every workaround names the hardware fact it works around.

`DEAD_ZONE_MIN_PWM = 30` should read *"below this the motors stall — measured with motor_balance_test, 2026-09"*. Then the next reader — including Egemen in March — can tell a real finding from a guess somebody made once.

20.6 The safeguard

`LEGACY/` is not deleted until the new car beats it on the practice track.

It stays runnable all year as the benchmark. This turns "start fresh" from a gamble into a measured migration: at every phase, "is the new one actually better?" is answered by a run rather than an opinion.

It also covers the realistic risk. A full rewrite in one school year, on a team where one person does most of the work, is ambitious. If the rewrite stalls in March, there is still a car.

20.7 Run the cheap experiment *before* committing to the rewrite

The full-code audit changed what we know. The failure was **not** bad algorithms — it was unfinished calibration (3.1, 3.2) and unconnected features (3.5). Those are days of work, not months.

So do this first, in the first week back:

1. Run `calibrate.py`, fix `PERSP_SRC` for 800×680, record the new corners.
2. Run `motor_balance_test.py`, record the trim values — and remember the bugs in 3.3 will activate the moment those values stop being `1.0`, so fix those two lines at the same time or the test will mislead you.
3. Drive it on the practice track and read the `logger.py` stability report — it already prints lost-lane percentage, mean error, standard deviation and FPS.

If the car suddenly follows the road, that is enormously valuable information. It means the codebase was one calibration away from working, and the rewrite becomes a *controlled* exercise in understanding rather than a rescue. If it still wanders, you have eliminated the two most likely causes for the price of an afternoon.

This does not reverse the rewrite decision — the reasons in 20.1 stand regardless. It changes how much of `lane.py` you should be in a hurry to replace.

20.8 Defects the rewrite must not reproduce

Found in the full audit. Each is a specific thing to design out, not just avoid:

Defect	Design rule for the new code
`PERSP_SRC` stale after a resolution change (3.1)	Resolution-dependent constants must be **derived from `WIDTH` / `HEIGHT`**, or validated against them at startup and refused if inconsistent. `ayar.py` already has validation as its job
Trim applied in two places, wrong wheel (3.3)	Trim belongs in exactly one layer — the motor driver — and is selected **by wheel identity, never by the sign of a number**
`sign_type` consumed but never produced (3.5)	The event dictionary needs a **single declared schema**. A consumer reading a key no producer writes should fail loudly at startup, not silently return `None` for a whole season

Defect	Design rule for the new code
Errors caught, motors left running	`main.py` 's loop catches exceptions, prints, and **continues without braking** — up to 30 consecutive failures before it exits. At 25 FPS that is over a second of blind driving at the last commanded speed. The new loop **brakes first, then logs.** This is the `CLAUDE.md` motor-safety rule and the legacy code does not honour it
Crosswalk uses a fixed white threshold while lanes adapt	`events.py` calls the non-adaptive `WHITE_HSV_LOW/HIGH` while `lane.py` picks a DARK / NORMAL / BRIGHT profile by scene brightness. Under venue lighting these disagree. **One brightness decision per frame, shared by every detector**
Nine overlapping tuning tools	One calibration GUI, owned by Tuna (section 9)

20.9 What the audit says to keep — confirmed by reading, not assumed

- **`lane.py` in full.** Bird's-eye warp, CLAHE on the L channel, adaptive white profiles, **column-continuity weighting** (a genuinely clever reflection filter — a lane line is vertically continuous in bird's-eye view, a glare spot is not), near/far weighted error, lane-position memory with a narrowed search window. The memory (`LANE_MEMORY_FRAMES = 25`) is also what already handles dashed centre lines, which closes most of question 8.
- **`events.py` debouncing** (`EVENT_DEBOUNCE_FRAMES = 6`), the crosswalk band/gap/width checks, and the parking guard that requires the red blob to be *near* **and** no orange car in the same frame.
- **`controller.py`** as described in 3.0.
- **The two-stage approach states.** `YAYA_YAKLAS` → `YAYA_GECİDİ` (see it far, close slowly, stop at 30 cm) is better than a single trigger and should survive the rewrite.
- **`logger.py` is better than first credited.** CSV export plus a stability report with FPS, lost-lane percentage, mean, standard deviation and peak error. `kayit.py` should *extend* this, not replace it.

21. Documentation audit — what the old docs claimed vs what exists

Performed 1 August 2026 against every file in `LEGACY/`. This section exists because the **documentation, not the code, is what sank the May run.**

21.1 How to reproduce this check in ten seconds

Extract every `ALL_CAPS` identifier, `method_name()` and `file.ext` reference from a document, then check each against the source. Anything a document names must be findable:

```
grep -rn "SHARPNESS_THRESHOLD_HIGH" *.py     →  no results  →  fabricated
```

Every fabrication below was found this way. It does not require reading anything carefully, and it should be run on any document before the team relies on it — **including this plan.**

21.2 Tier 1 — fabricated

`BASLA_BURADAN.txt`, `DOSYALAR_GUNCELEME_DURUSU.txt`, `DEGISIKLIKLER_OZET.txt`, `IMPLEMENTATION_SUMMARY.txt`. All dated 2026-04-25, eleven days before the competition.

Constants described in detail that exist in no file:

`ADAPTIVE_HSV_ENABLED` · `ADAPTIVE_PD_ENABLED` · `HSV_BRIGHT_THRESHOLD` · `HSV_DARK_THRESHOLD` · `MOTOR_DEAD_ZONE` · `MOTOR_DEAD_ZONE_BOOST` · `DEAD_ZONE_BOOST` · `SHARPNESS_THRESHOLD_HIGH` · `SHARPNESS_THRESHOLD_MED`

The adaptive-HSV thresholds are hardcoded inline in `lane.py` as `100` and `200`. **There is no sharpness logic anywhere in the project, in any version** — that feature was invented whole.

Methods described with their behaviour that do not exist:

`_get_adaptive_hsv()` · `_get_adaptive_kp()` · `_get_adaptive_kd()` · `_apply_response_mapping()` · `_apply_dead_zone_compensation()`

The last is the informative one — the real method is `_apply_dead_zone_pair()`. The docs describe a **renamed predecessor**, so they were never re-checked after the code changed.

Files referenced that do not exist — ten:

`FINAL_DELIVERY.md` · `OPTIMIZASYONLAR_DETAYLI.md` · `OZET_VE_DEGISIKLIKLER.md` · `README_OKUMA_SIRASI.md` · `YARISMA_GUNU_REHBERI.md` · `DEGISIKLIKLER_OZET.md` · `HIZLI_REFERANS.md` · `OPTIMIZASYONLAR_UYGULAND.md` · `TEST_OPTIMIZASYONLAR.py` · `start.sh`

`BASLA_BURADAN.txt` instructs the reader to open three of these **first**, and names `FINAL_DELIVERY.md` as the single most important file in the project.

Values that are simply wrong:

Claimed	Actual
`KP: 0.40 → 0.60`	`KP = 0.30`
`KD: 0.10 → 0.44`	`KD = 0.45`
`KD_LARGE_ERROR_MULT ×1.5`	`1.2`
Derivative cap `clip(-600, 600)`	`DERIV_CAP = 150`
config / lane / controller = 252 / 205 / 153 lines	293 / 253 / 184
`events.py` 600+ lines, `main.py` 900+ lines	406 and 627

Measurements presented as achieved, never taken:

- "±40 px → ±15 px deviation, +62.5%"
- "24–26 FPS → 28–30 FPS" — **inverted**; adding per-frame CLAHE and extra branching can only reduce frame rate
- "Motor responsiveness 1.2 s → 0.3 s, +75%"
- "+85–95 puan", itemised per feature (`Adaptif HSV → +30p`). MEB scoring does not assign points to code changes at all

`BASLA_BURADAN.txt` signs off with `📍 DOSYA KONUMU: /mnt/user-data/outputs/` — an AI sandbox path, copied verbatim into the team's repository.

21.3 Tier 2 — `LEGACY/CLAUDE.md` , honest but drifted

This is the important contrast. `CLAUDE.md` is **broadly accurate**: its descriptions of `lane.py` 's pipeline, the debouncing in `events.py` , `logger.py` 's behaviour, the `gpiozero` wrapper and the `GG` / `EZ` / `SPACE` start path all check out. It even correctly flags the double-applied trim.

Its four errors are drift, not invention:

1. `KIRMIZI_ISIK` **does not exist** in any `.py` file. It is listed as a transient state.
2. **Three real states are missing** — `YAYA_YAKLAS` , `HEMZEMIN_YAKLAS` , `CIKMAZSOKAK` . The two-stage approach behaviour, the best design in the state machine, is undocumented.
3. It calls `yol_takip.py` an abandoned experiment; it is a working lane-only build (20.3b).
4. It records the double trim as "*intentional in current code*", which launders a bug into a design decision.

For the record, the eleven states that actually exist: `BEKLIYOR` , `SURUYOR` , `YAYA_YAKLAS` , `YAYA_GECİDİ` , `HEMZEMIN_YAKLAS` , `HEMZEMIN` , `TUMSEK` , `SOLLAMA` , `PARK` , `CIKMAZSOKAK` , `PARK_TAMAM` .

21.4 Tier 3 — the technical document (judge-facing)

Covered in section 14. `Python 3.19` (does not exist), `RPi.GPIO` (the code uses `gpiozero`), `16850 Li-Po` (should be 18650 Li-ion), a pin table listing pin 11 twice with contradictory meanings, front/rear labelling for a left/right car, `OpenCV 4.2` , and *bölge tamamlama* missing entirely.

21.5 Two different diseases — and only one of them is "telephone"

Drift (telephone). Information that was true once and went stale: the `_apply_dead_zone_pair` rename, `CLAUDE.md` 's state list, the two forked configs, the technical document's pin table. Caught by **re-reading**. A single source of truth fixes it, which is what this plan is for.

Confabulation. Claims that were never true in any version: the sharpness thresholds, the five phantom methods, the ten phantom files, the invented measurements. You cannot degrade a signal that was never transmitted. This is an assistant writing a completion report for work it *planned* rather than *verified* — the headers give it away: `Durum: KAPSAMLICA GÜNCELLENDI` , `✅ TÜM OPTİMİZASYONLAR UYGULANMIŞTIR` . Caught only by **checking claims against the artifact**.

Why it survived. The reports are not fiction. Six of their headline features genuinely exist — adaptive HSV, CLAHE, dead-zone compensation, speed-dependent trim, dynamic gain, derivative capping are all really in the code. So the *architecture* claims are true and every *specific* number, name, file and metric is invented. Check two or three claims, find them correct, stop checking. A document that was wholly wrong would have been discarded in a minute.

Consequence for this project's stated fix. Compacted histories and one source of truth solve drift completely and do **almost nothing** about confabulation. Perfect memory and confident self-reporting are independent problems. That gap is what rule 7 in `CLAUDE.md` now closes.

21.6 A bug this audit surfaced

`LOG_DURATION_SEC = 120` , but the race limit is **240 seconds**. The only telemetry the car had stopped recording **halfway through the run**. If the failure happened in the back half of the course, the log was already closed. This is part of why May produced no evidence, and `kayit.py` must run for the whole race plus margin.

21.7 Actions

- **Delete `BASLA_BURADAN.txt`, `DOSYALAR_GUNCELEME_DURUSU.txt`, `DEGISIKLIKLER_OZET.txt` and `IMPLEMENTATION_SUMMARY.txt`.** They are confidently wrong, and nothing in them survives into this plan except this section. Do this only after 20.4 mining is finished.
- **Correct the four `CLAUDE.md` drifts** if `LEGACY/CLAUDE.md` is kept for reference.
- **Fix the technical document** before the next application (section 14).
- **Run 21.1 against this plan** before the team relies on it. Much of it was written by assistants, including this section.

22. SUBIRU v2 — the enforcement design

Answers the question left open in `readme.txt` and `CLAUDE.md` since the beginning: what should "enforce" actually mean.

22.0 What this system is optimising for

Stated first, because it decides every trade-off below.

This project's value to the people in it is **going places together and finishing the course** — not maximising a score. That has three concrete consequences:

- **Qualifying reliably beats peak performance.** A car that completes every round is worth more than one that occasionally scores brilliantly and occasionally does not finish (section 2.1: rounds are summed, and the time bonus needs a completed course).
- **Teammates staying beats teammates producing.** A member who returns next season is worth more than an extra task closed this one.
- **Therefore this system reports, it does not prosecute.** Where a design choice trades accountability against someone wanting to keep turning up, it goes the second way. See 22.3b and the framing rule there.

22.1 The goal is a record that is TRUE, not a record that is FULL

This is the whole design. Optimise for "everything marked done" and you rebuild `BASLA_BURADAN.txt`, which was a *perfect* completion record — every box ✅, nothing verified (section 21).

There are two separate failures and they need different mechanisms:

Failure	Cause	Mechanism
Work done, never recorded	Recording is pure overhead	**Derive status from artifacts** (22.3)
Work recorded, never done	Nothing checks the claim	**Require evidence** (22.4)

SUBIRU currently addresses neither, which is why `tasks.json` is `[]`.

22.2 Extended `tasks.json` schema

Existing fields stay: `id`, `title`, `owner`, `status`, `depends_on`, `created_at`, `updated_at`, `notes`. Added:

Field	Type	Purpose
`kind`	`code` | `kalibrasyon` | `donanim` | `test` | `idari`	Decides what counts as evidence
`evidence`	`null` | object	See 22.4. `null` means not done, whatever the status says
`phase`	`0` – `5`	Which plan phase (section 12); drives gating
`files`	list of paths	What this task touches — powers git harvest and stall detection
`fresh_until`	`null` | ISO date	Calibration expiry (22.5)

Migration is free. `storage.py:load_tasks` builds tasks with `Task(**item)`, so new fields with defaults load old records unchanged — and `tasks.json` is currently empty anyway. Note the reverse is not true: renaming or removing a field breaks `Task(**item)` with a `TypeError`. Add, never rename.

22.3 Harvest status from git — stop asking people to type

`tasks.json` is empty because updating a dashboard gives nothing back. Fix the incentive rather than the discipline: derive what git already knows.

For each task with `files`, read the last commit touching them — hash, author, date. The dashboard then shows "*`Lane.py`: last touched 12 days ago by Egemen*" with nobody typing anything. **Working updates the record as a side effect.**

This also powers **stall detection**, which targets this project's actual signature — starting competently and stopping one step short (the sign model, the balance tool, the perspective warning). Rule: `status == in_progress` **and** no commit touching `files` for more than `STALL_DAYS` → flag as *started, stalled*.

22.3b Git cannot see hardware work — do not let the dashboard punish it for that

This is a correction to the design above, and it matters more than the feature it corrects.

Git observes commits. Tuna's job is chassis, wiring, mounting, measuring a deviation with a ruler, photographing signs, driving the car during tests. **None of that produces a commit.** Applied naively, 22.3 would show him as permanently idle while he is doing precisely the work he owns, and then flag him as *stalled* for it — an enforcement system quietly building a weekly case against the person it is least equipped to observe.

Two fixes, and the first is far better than the second.

1. Route his output into the repo so git *can* see it. This makes his work count, rather than merely not counting against him:

- Calibration values land in `kalibrasyon.json` — a real file, committed by him, from his own machine (which is why his SSH key is in Phase 0).
- Sign training photos land in a repo folder.
- Track footage lands in `klipler/` with a manifest, even when the video itself stays out of git.
- Hardware changes get a photo committed alongside the note.

After this, most of what he does *is* visible in the log, attributed to him, permanently. That is the difference between a division of labour and a bottleneck.

2. Where no signal exists, say so — never infer idleness. Physical assembly genuinely produces nothing automatic. For `kind: donanim`, and for any task whose `files` is empty, the dashboard shows **"not observable from git"**, never *stalled*. Absence of evidence is not evidence of absence, and a tool that confuses the two will be wrong about the same person every single week.

Framing rule. A stall flag is information for the person who owns the task, not an accusation delivered to the room. The goal of this system is a team that still wants to turn up next season — a dashboard that makes someone feel monitored will cost more than any task it recovers.

22.4 KANIT — evidence, typed by task kind

A task may not reach `done` without an `evidence` object:

```
evidence = {
  type:        matches the task's kind
  value:       see table
  recorded_at: ISO timestamp
  recorded_by: who or what attached it
}
```

`kind`	`value` is	Checkable by
`code`	commit hash	`git cat-file -e <hash>` — refuse if it does not resolve
`kalibrasyon`	the measured number, its unit, and where it was measured	Human, but the number must exist
`donanim`	path to a photo	File exists
`test`	path to the run log	File exists and is non-empty
`idari`	a note naming who confirmed it	Human

"I did it" stops being an accepted answer.

This would have caught the trim failure exactly. There was no number to enter, because pasting `LEFT_TRIM = 0.95` into a config that reads four other names produces nothing recordable. The absence of evidence *was* the bug (section 20.3e).

22.5 Done expires — calibration decays

The mechanism most specific to this project. Lighting changes, motors wear, the camera gets knocked. "Tune white HSV" is never done permanently — it is done **as of a date, in a room**.

So `kind == "kalibrasyon"` tasks get `fresh_until = evidence.recorded_at + KALIBRASYON_OMRU` and revert to a visible **stale** state when it passes. Something calibrated in September reads red by March.

The payoff is walking into Antalya able to see, at a glance, exactly which numbers were last measured in a school corridor seven months earlier. The existing `STALE_DAYS = 3` row-tint is the same idea; this aims it at the thing that actually rots.

22.6 Gate phases, not just tasks

`storage.py:can_advance` blocks task→task through `depends_on`. Aim it one level up: section 12 already defines an **exit test per phase**.

Make each exit test a task of `kind: test`, and refuse to open any Phase N task until Phase N−1's exit test carries evidence. That turns this plan from a document someone is supposed to remember into the thing the tool enforces.

22.7 `kontrol.py` files its own evidence

The pre-flight checklist (management suggestion #2) and the tracker should be one system, not two things to maintain.

`kontrol.py` writes a JSON result; SUBIRU reads it and attaches results to matching tasks automatically — green closes the calibration tasks, red reopens them. The pre-race check then cannot be silently skipped **and** nobody has to update a dashboard. One command does both.

22.8 Who may mark done

Humans, and passing scripts. **Never an assistant.** An LLM may create tasks, propose evidence and fill in `files`, but the `done` transition requires a `recorded_by` that is a person or `kontrol.py`.

This is a convention rather than something enforceable in code, and it is stated anyway, because four documents marked this entire project complete and none of it was (section 21).

22.9 Build it in stages — or it becomes the fourth abandoned feature

The realistic risk is that SUBIRU v2 joins the sign model, the balance tool and the perspective warning as something begun well and left one step short. So each stage must be independently useful and shippable alone:

Stage	Contents	Value if you stop here
1	`kind` + `evidence`, refuse `done` without it	The core benefit. Half a day
2	Git harvest: last-touched, stall flags	Dashboard populates itself
3	Freshness / expiry	Pre-competition calibration is visible
4	Phase gates from section 12	The plan enforces itself
5	`kontrol.py` integration	One command, both systems

Stage 1 alone is worth more than the other four combined. Do not start stage 2 before stage 1 is in use with real tasks in it.

22.10 Two things this cannot do

It cannot make anyone work. No software can. What it does is make the truth cheap to see, so a conversation with a teammate is about a red row and a missing number instead of about effort and intentions. That is a much easier conversation to have and a much harder one to argue with.

It cannot be imposed unilaterally. Enforcement that one person designs and the other never agreed to is just nagging with extra steps. Tuna should see 22.4 and 22.5 and say yes to them *before* they are built — and the fact that his contributions become permanently attributable is a point in his favour, not a trap.

22.11 Hosting SUBIRU on the VPS — after stage 2, not before

A dashboard only Egemen can see enforces nothing. SUBIRU currently runs from `run.bat` on one laptop at `127.0.0.1:5057`, which means Tuna sees the board only when Egemen is online and has it running. Hosting it on the team VPS fixes that, and visibility is the entire point of section 22.

Two conditions, both firm:

Wait for stage 2. An empty board hosted publicly is still an empty board. Host it once git harvest (22.3) makes it populate itself — otherwise the first thing Tuna ever sees is a blank page, and he will not come back.

Fix the security first — this is real, not theoretical. `subiru/app.py` line 118 runs `app.run(debug=True, ...)` and line 18 sets `app.secret_key = "subiru-dev-secret"`. Flask's debugger on a public address is a **remote code execution hole**, and a hardcoded key makes sessions forgeable. Fine on localhost; not fine on an IP. Before it leaves the laptop: `debug=False`, a real secret from the environment, and a proper WSGI server rather than the development one.

Note this does not conflict with `CLAUDE.md`'s no-gatekeeping rule. That rule is about not making *teammates* prove who they are. It is not an argument for leaving a debugger open to the whole internet.

22.12 What the VPS must never do

The car does not talk to it. Rule 2.3 requires Bluetooth, Wi-Fi and RF **off** during a run, and detection of an active radio is immediate disqualification (section 2.2). So no telemetry streaming, no live monitoring, no phoning home. This is not a design trade-off, it is a rule.

`kayit.py` writes to the SD card precisely because the radio must be dead. Uploading logs **after** a run, from the pits, is fine and encouraged. During a run, never.

22.13 First tasks to seed

Phase 0's bullets (section 12), each with `kind`, `phase: 0`, and `files` filled in. Committing `LEGACY/` is task #1 and its evidence is a commit hash — which makes the very first use of the system a demonstration of it.